

**Deployment and Optimization of Large
Language Models and Retrieval-Augmented
Generation Pipelines on High-Performance
Computing Systems**

Bachelor of Science in Computer Science

Author: Nikos Kalogirou

Supervisor: Dr. Christodoulos Stylianos

Declaration

I Nikos Kalogirou with Reg. no. 20231063 in full knowledge of the consequences of plagiarism, hereby solemnly declare that the present thesis titled: *Deployment and Optimization of Large Language Models and Retrieval-Augmented Generation Pipelines on High-Performance Computing Systems* is strictly my own original work and all the sources I have used have been duly and accurately acknowledged in the bibliographical references. The passages where I have used material from other authors are clearly documented in the text and the relevant references are included in the Works Cited section.

Acknowledgments

I would like to thank Dr. Christodoulos Stylianou, my supervisor, for his guidance, constructive feedback, and support during the development of this project. His advice helped me focus the work, improve the technical implementation, and structure the thesis.

I would also like to acknowledge the use of the Cyclone high-performance computing facility at The Cyprus Institute, which supported the HPC execution stage of this work.

Furthermore, I would like to express my appreciation to the Department of Computer Science and Engineering at European University Cyprus for providing the academic environment and resources that supported this work. I am grateful to the lecturers and staff who contributed to my studies and helped me build the knowledge needed to complete this project.

Finally, I would like to thank my family and friends for their encouragement, patience, and support during the preparation of this thesis. Their motivation and understanding were important throughout the research, implementation, and writing process.

Abstract

Large Language Models (LLMs) are becoming increasingly popular in many different Natural Language Processing (NLP)-related areas (such as Question Answering (QA), summarization and information retrieval). However, LLM on its own may provide unsupported answers if the required information exists at a remote location or is specific to a particular area of study. Retrieval-Augmented Generation (RAG) solves this problem by first finding all relevant parts of a document collection and then making these available to the LLM as context prior to generating the final answer.

This thesis examines the optimization and deployment of RAG pipelines through a comparative analysis of six controlled pipeline configurations: (i) in-memory baseline (ii) a Milvus vector database pipeline (iii) a Milvus vector database pipeline with tuning (iv) dense retrieval with reranking (v) hybrid retrieval without reranking (vi) hybrid retrieval with reranking. For all six of the pipeline configurations described above, there employed the same synthetic enterprise document collection, same set of five scenarios, same log output format and same evaluation procedure. Thus, the only difference among the six pipeline configurations is in their retrieval strategy.

The main comparison uses manual answer score to measure answer correctness and for the retrieval it uses the F1 score to measure how well precision and recall are balanced when the retrieved documents are compared with the expected supporting evidence.

The local results showed that no single pipeline achieved the optimal F1 score for both the retrieval and the answers. The pipeline that utilized in-memory data had the highest manual answer mean score, the Milvus based pipeline had the highest average retrieval F1. In some cases reranked search provided better results than a simple first pass, however there was an increase in time to retrieve. Overall these are good indicators that simply having a "better" search engine does not guarantee getting the most accurate answers, as well as all components need to be evaluated to determine if they add sufficient benefits to warrant their costs.

The High-Performance Computing (HPC) evaluation compared the six workflows run locally to the HPC environment utilizing only time based comparisons. Each of the six workflows experienced a reduction in total runtime

when running on the HPC platform. An overall average speed-up of approximately $5.05\times$ times was observed by each of the six pipelines. The greatest total speed-up was seen for the basic pipeline configuration. Generation time improvement existed in each of the configurations; however, the improvement in retrieval time was inconsistent among the Milvus based configurations. Some of these configurations resulted in longer retrieval times than their respective base configuration when run on HPC indicating that the cost associated with interacting with services, accessing storage and retrieving data from a vector database may be higher than expected when performing short retrieval operations.

The overall framework that has been created in this thesis is a structured local analysis of RAG variants using timing-based performance evaluations, quality assessments based on retrieval characteristics, quality assessments of answers generated by models, and a fully reproducible experiment workflow to run and log experiments in both local and HPC environment. The full training of models, the fine-tuning of models, the compression of trained models, as well as large scale multi-node benchmarking are beyond the scope of this research.

List of Abbreviations

NLP	Natural Language Processing
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
HPC	High-Performance Computing
GPU	Graphics Processing Unit
CPU	Central Processing Unit
LoRA	Low-Rank Adaptation
KV	Key-Value
DPR	Dense Passage Retrieval
BM25	Best Matching 25
FSDP	Fully Sharded Data Parallel
TGI	Text Generation Inference
MMLU	Massive Multitask Language Understanding
HELM	Holistic Evaluation of Language Models
GPFS	General Parallel File System
I/O	Input/Output
RAGAS	Retrieval Augmented Generation Assessment
ROUGE-L	Recall-Oriented Understudy for Gisting Evaluation - Longest Common Subsequence
BERTScore	Bidirectional Encoder Representations from Transformers Score
AWS	Amazon Web Services
vLLM	Virtual Large Language Model
FP32	Single-Precision Floating-Point (32-bit)
FP16	Half-Precision Floating-Point (16-bit)
INT4	4-bit Integer Precision
INT8	8-bit Integer Precision
GPTQ	Generalized Post-Training Quantization

AWQ Activation-aware Weight Quantization

KV-cache Key-Value Cache

KILT Knowledge Intensive Language Tasks

MiniLM Minimally Distilled Language Model

QA Question Answering

SLA Service Level Agreement

Sev-1 Severity 1)

Contents

List of Figures	6
List of Tables	7
1 Introduction	8
1.1 Introduction	8
1.2 Motivation	9
1.3 Problem Definition	10
1.4 Research Objectives and Scope	11
1.5 Research Questions and Hypotheses	12
1.6 Research Methodology Overview	14
1.7 Expected Contributions	15
1.8 Structure of the thesis	15
1.9 Summary	16
2 Background and Theoretical Foundations	18
2.1 Introduction	18
2.2 Large Language Models (LLMs)	18
2.2.1 Overview	18
2.2.2 Transformer Architecture	19
2.2.3 Development Paradigms and their Effect on LLM Performance	22
2.3 Retrieval-Augmented Generation (RAG)	23
2.3.1 Concept and Motivation	23
2.3.2 Architecture and Workflow	24
2.3.3 Retrieval Techniques	25
2.4 High-Performance Computing (HPC) Systems	27
2.4.1 Overview	27
2.4.2 Architecture and Components of HPC	27
2.4.3 Parallelism, Caching, and Batching in RAG Workloads	28
2.4.4 Job Scheduling and Resource Management	29

2.4.5	Containerization	29
2.4.6	Performance Optimization	29
2.5	Evaluation Metrics in Model Deployment	31
2.5.1	System Performance Metrics	31
2.5.2	Retrieval Quality Metrics	32
2.5.3	Answer Quality Metrics	32
2.5.4	Local versus HPC Comparison Metrics	34
2.6	Summary	34
3	Literature Review	35
3.1	Introduction	35
3.2	LLM Deployment Approaches	35
3.2.1	Local and Cloud-Based Deployments	35
3.2.2	HPC-Based Deployments	36
3.2.3	Model Serving Frameworks	37
3.3	Optimization Techniques for LLM Inference	38
3.3.1	Quantization and Pruning	38
3.3.2	Knowledge Distillation	39
3.3.3	Parallelization and Sharding	39
3.3.4	Caching and Memory Optimization	39
3.4	RAG Systems and Optimization Strategies	40
3.4.1	Existing Frameworks	40
3.4.2	Retrieval Efficiency and Hybrid Approaches	40
3.4.3	RAG on HPC Environments	41
3.5	Benchmarking and Evaluation Studies	41
3.6	Positioning of This Thesis	42
3.7	Summary	42
4	Methodology	44
4.1	Introduction	44
4.2	Research Framework and Design	44
4.3	Dataset and Input Documents	45
4.4	System Architecture	45
4.5	Implemented RAG Pipelines	47
4.5.1	Baseline RAG version	47

4.5.2	Version Using a Vector Database RAG	47
4.5.3	Tuned Version Using a Vector Database RAG	48
4.5.4	Reranking RAG	48
4.5.5	Hybrid Retrieval RAG	48
4.5.6	Hybrid Retrieval With Reranking RAG	49
4.5.7	Measured Variables	49
4.6	Evaluation Framework	50
4.6.1	System Performance Metrics	50
4.6.2	Retrieval Quality Metrics	51
4.6.3	Answer Quality Metrics	51
4.6.4	Visual Analysis	51
4.6.5	Cross-Environment Comparison	52
4.7	Reproducibility and Documentation	52
4.8	Summary	52
5	Experiments	54
5.1	Introduction	54
5.2	Experimental Setup	54
5.2.1	Software Environment	54
5.2.2	Models and Services	55
5.2.3	Input Corpus	56
5.2.4	Configuration and Parameters	56
5.3	Deployment Stages	57
5.3.1	Local Execution Stage	57
5.3.2	HPC Execution Stage	57
5.4	Experiment Scenarios	58
5.4.1	Scenario-Based Query Execution	58
5.4.2	Baseline RAG Experiment	60
5.4.3	Vector Database RAG Experiment	60
5.4.4	Tuned Vector–Database RAG Experiment	60
5.4.5	Reranking Experiment	60
5.4.6	Hybrid Retrieval Experiment	61
5.4.7	Hybrid Retrieval with Reranking Experiment	61
5.5	Benchmarking and Data Collection	61
5.6	Data Validation	62

5.7	Summary	62
6	Results and Discussion	63
6.1	Introduction	63
6.2	Results of the Local Runs	63
6.2.1	Local Timing Comparison	64
6.2.2	Local Retrieval and Answer Quality	65
6.3	HPC Deployment Results	67
6.4	Local versus HPC comparison	68
6.5	Effectiveness of Retrieval-Level Optimization Techniques . .	69
6.6	Research Questions and Hypotheses	71
6.7	Comparison with Literature	72
6.8	Trade-offs and Limitations	73
6.9	Summary	74
7	Conclusion and Future Work	76
7.1	Introduction	76
7.2	Summary of Findings	76
7.3	Key Contributions	77
7.4	Limitations	78
7.5	Future Work	79
7.6	Final Remarks	80
	Appendices	86
	Appendix A: Experimental Artifacts and Reproducibility Checklist	86
	A.1 Project Artifact Map	87
	A.2 Pipeline Summary	88
	A.3 Scenario Set	89
	A.4 Logged Fields	89
	A.5 Reproducibility Checklist	90
	Appendix B: Evaluation Rubric and Supporting Evidence Guide .	91
	B.1 Manual Answer Score	91
	B.2 Supporting Document Rules	91
	B.3 Deterministic Evaluation Outputs	92
	Appendix C: Supplementary Figures	93

Manual	95
Installation Manual	95
Purpose and Scope	95
Required Software for Local Execution	95
Local Installation Steps	95
Milvus Management	96
HPC Installation and Service Preparation	97
User Manual	98
Project Workflow	98
Running the Local Experiments	98
Running the HPC Experiments	99
Manual Ground-Truth Evaluation	99
Running Deterministic Evaluation	100
Generating Figures	100
Result Validation Checklist	100
Common Issues	101

List of Figures

1.1	Overall thesis workflow showing the local deployment stage, the later HPC deployment stage, and the final comparison between RAG variants and execution environments.	14
2.1	Transformer encoder–decoder architecture	20
2.2	Multi-head attention mechanism [1].	21
4.1	High-level RAG workflow used by the implemented notebooks: user query, retrieval over indexed documents, context construction, and answer generation.	46
6.1	Average local timing breakdown for indexing, retrieval, and generation across the implemented RAG variants.	64
6.2	Answer-quality metrics for the local RAG variants, including manual score, ROUGE-L, BERTScore, and semantic similarity.	65
6.3	Retrieval-quality metrics for the local RAG variants, comparing hit rate, precision, recall, and F1.	66
6.4	Manual answer score by implementation and scenario. Darker cells indicate stronger manually evaluated answers. .	67
6.5	Multi-metric profile of the local RAG pipelines, comparing manual score, BERTScore, semantic similarity, retrieval recall, and retrieval precision.	67
6.6	Mean total runtime for each RAG pipeline in the local and HPC environments.	69
6.7	Speed-up heatmap comparing local and HPC execution across indexing, retrieval, generation, and total runtime. . . .	69
6.8	Runtime component breakdown for local and HPC execution.	71
7.1	Mean indexing time by pipeline and execution environment.	93
7.2	Mean retrieval time by pipeline and execution environment. .	93
7.3	Mean generation time by pipeline and execution environment.	94
7.4	Total speed-up factor for each RAG pipeline when comparing local and HPC execution.	94

List of Tables

2.1	LLM development paradigms and their relevance to this thesis.	23
2.2	Metric groups used in the thesis evaluation framework. . . .	33
3.1	Summary of related work areas and their relevance to this thesis.	43
4.1	Implemented RAG pipeline variants and included components.	49
6.1	Summary of local evaluation results averaged across the five scenarios.	63
6.2	Local versus HPC timing speed-up summary.	68
7.1	Summary of research-question answers and hypothesis outcomes.	77
7.2	Main artifacts used to reproduce and document the thesis experiments.	87
7.3	Implemented pipeline variants and their corresponding notebooks.	88
7.4	Scenario questions used across all six pipeline implementations.	89
7.5	Manual answer scoring rubric used in the thesis evaluation. .	91
7.6	Recommended notebook execution order.	98

Chapter 1 – Introduction

1.1 Introduction

Large Language Models (LLMs) were developed as a result of advancements in deep learning that allow them to read, understand, and create large amounts of natural language. The development of these LLMs has been directly linked to an architectural advancement known as the Transformer. This architecture allows for the ability to model sequences based upon their relevance (attention) and allowed efficient training on larger than ever before data sets, in parallel [2]. With increases in both the amount of data available, computing power and model size, LLMs have achieved impressive results across a variety of tasks including question answering, generating summaries, coding, and extracting information [3, 4]. These capabilities have provided several opportunities for their use in various types of applications, however, with this come significant technical barriers for deployment LLMs due to the issues associated with memory usage during processing, processing speed/latency for real-time operations, reliance on other software packages, and ensuring reproducible results.

One technique used to improve both the utility and credibility of Large Language Model (LLM) systems is called Retrieval-Augmented Generation (RAG). The RAG system takes advantage of a retrieval mechanism that pulls in relevant data from an outside body of documents, and then incorporates these retrieved data into the final input (i.e. model prompt), before generating an answer [5, 6]. Therefore, as opposed to answers being generated using only what is contained within the model's parameterized knowledge base, the answers are now supported by some form of evidence pulled directly from the outside world. In addition to supporting domain adaptation, RAG can be done without additional model training or full weight-fine-tuning costs [7].

At the same time as producing answers, RAG systems aren't simply single-model systems. Document loading, chunking, embedding creation, indexing, retrieval (with optional reranking), prompt creation, answer creation, and log creation (i.e. logging results) occur during the operation of a typical RAG system. Since each of these components can influence both the quality of the final answer created by the system and the amount of time it takes to

execute the entire RAG system, in addition to determining if an answer was generated, a successful RAG project will also need to determine how different retrieval strategies impact latency, evidence quality, answer accuracy, and reproducibility.

This thesis also relates to High-Performance Computing (HPC) as an appropriate area of deployment. Many HPC systems have the characteristics of shared compute resources, job schedulers, execution environments that are based on containers, large-scale storage and scalable hardware in order to support high computational workloads [8–10]. The HPC stage of this experiment builds upon the previous work done using notebooks and extends it to a much larger and more complex environment. Therefore, comparisons can be made with regard to timing behaviour when identical RAG workflows are run in different environments.

1.2 Motivation

The primary purpose behind creating this project is to address the increasing demand for Large Language Model (LLM) applications that use private domain-specific data or constantly evolving data. A single, self-contained language model will be able to generate coherent responses; however, it will likely lack immediate access to the specific documentation required to perform the desired function. Therefore, the model could potentially create responses that seem valid, yet would be unsupported by existing documented proof. This is where RAG can help mitigate some of these limitations by allowing the system to search a previously established body of knowledge, retrieve relevant pieces of text, and pass these retrieved texts into the language model as supplementary input before generating an answer.

Although the idea of RAG seems relatively simple, the performance and speed of a full RAG solution heavily depends on how each piece of the solution is implemented. For example, building a very basic-in-memory retriever is easily accomplished and works well when working with smaller bodies of text. However, using a basic in-memory retriever will not provide the same level of persistence or deployment options as a vector database. For instance, while Milvus (a vector database) provides much better persistence options and a much more viable platform for working with large-scale retrieval requirements than an in-memory basic approach, it comes at an added expense through both set up costs and indexing costs. Although reranking

can improve retrieval quality by reordering retrieved items, it adds another model to the pipeline and increases overall retrieval latency. Similarly, hybrid retrieval allows for the combination of semantic and keyword-based retrieval methods; however, it also adds increased complexity to the overall retrieval method.

This explains why this experimental comparison is conducted between in-memory retrieval, vector-database retrieval, vector-database retrieval with optimized chunking, dense retrieval with reranking, hybrid retrieval and finally hybrid retrieval with reranking (a combination of the latter two). Reproducible workflow is required because practical RAG solutions comprise numerous service providers, models, notebooks, configuration files, result logs, evaluation scripts, and generated images for the results.

1.3 Problem Definition

The general problem addressed by this thesis is how to easily and practically test different versions of a RAG pipeline under a deployment workflow that is fully reproducible. The primary issue is not whether RAG will improve general language-model applications as other studies have shown [5, 6], but instead, what effect do different retrieval and ranking strategies have on speed, the amount and quality of retrieved evidence, and the final quality of answers and how easy it is to reproduce these results.

A RAG pipeline contains several components whose behaviour must be coordinated. Documents must be loaded into memory and split into chunks; embeddings must be generated; indexes or document stores must be created; candidate passages must be retrieved and finally, using the selected context, the language model must generate an answer. Weaknesses at any stage can reduce the usefulness of the final system. For example, while a fast retriever may return documents that are irrelevant to the user’s query, these poorly chosen documents may lead to poor answers. Simultaneously, a re-ranking stage which may enhance the quality of the document choice may also result in significantly longer response times.

Therefore, the primary goal of this thesis is to study the implemented RAG pipelines both as exploratory software deployment and experimental conditions. It compares their output using exactly the same input documents, scenario queries, models and logging approach. This creates a controlled framework from which to evaluate the trade-off between performance and

output quality.

1.4 Research Objectives and Scope

The scope of this thesis is limited to retrieval-level RAG experimentation and reproducible deployment workflow design. The local experiments provide the empirical comparison, while the HPC workflow prepares for larger-scale execution.

The approach aims to use the same collections of documents, questions, open-source language and embedding models, as well as the same format for logging as much as possible throughout this thesis. This enables to measure the influence of different retrieval methods and deployment strategies independently. A baseline of behaviour using the developed notebooks during the local phase, is also established. During the HPC phase, the same files and scenarios are executed to compare the results obtained in the local and HPC environments within a common evaluation framework, with a focus on execution time.

To accomplish the objectives outlined in this thesis, the following five objectives will be met:

1. **Provide the theoretical and practical foundations for LLMs and RAG Systems.**

The thesis discusses the background of the necessary concepts to distinguish between model-level techniques (e.g. fine-tuning, training from scratch, quantization, pruning, adaptation) and the retrieval-level techniques that are being assessed in this work. Additionally, it introduces the concepts of RAG, Dense & Sparse Retrieval, Hybrid Retrieval, Reranking, Vector Databases, and HPC Deployment.

2. **Implement and assess the effectiveness of various RAG Retrieval Methods.**

The thesis describes the six methods for retrieval implementations. Each of the implementations is evaluated using standard retrieval metrics, such as hit rate, precision, recall, and F1, along with quality metrics for answers, for instance manual correctness scores, ROUGE-L scores, BERTScores, and semantic similarity.

3. **Determine the performance cost of the implemented RAG pipelines.**

This objective describes the timing (in seconds) that is measured using the "notebooks" and shared CSV-style result logs, for each of the RAG-pipelines built in this project. The metrics allow for a comparison of the costs of different processing methods such as in-memory-retrieval vs storage with a vector database, reranking retrieved candidates/passages from retrieval, sparse-dense retrieval combination, and varying the number of chunks.

4. Develop a workflow to compare local vs. HPC execution.

The local results that are initially collected, serve as a baseline for the HPC extension of this work. The HPC workflow is implemented using Virtual Large Language Model (vLLM)-based model serving for generation, embeddings, and reranking where applicable.

5. Create a reproducible valuation framework.

The thesis documents the software tools, execution procedures, scenario questions, logging fields, manual evaluation criteria, reproducibility scripts, and visualizations used during analysis.

This research limits its study to inference time LLM and RAG loads. This will enable the experiment to focus on retrieving quality answers as well as system performance. The scope does NOT include full model training or large-scale distributed training, model fine-tuning, model compression (quantization) or pruning, model distillation, and adapter based model optimization. These concepts have been referenced where necessary for a comparison in concept.

1.5 Research Questions and Hypotheses

The research questions are designed to remain within the experimental scope of the thesis. They focus on two measurable areas: the accuracy and answer quality of retrieval-level techniques, and the timing behaviour of the implemented workflows. The local experiments provide the first set of measurements, while the later HPC experiments repeat the same workflow to evaluate portability and deployment behaviour.

- **RQ1: How do different retrieval techniques affect the retrieval-accuracy and answer-quality of the implemented RAG pipelines?**

H1: Hybrid-retrieval will reach a better retrieval-coverage than dense-retrieval due to lexical-matching combined with semantic-similarity;

Reranking will make sure that retrieved passages are ordered well and thus increase top-k precision, answer-grounded-ness, and answer-correctness. It is assumed that the combination of hybrid-retrieval and reranking will lead to the best retrieval-quality results particularly for those queries which need an understanding of their semantic interpretation as well as the correct wording. Another expected noticeable improvement is the tuned Vector database implementation.

- **RQ2: What kinds of performance trade-offs are introduced through hybrid-retrieval and reranking?**

H2: The advantages at the level of retrieval are not cost-free in terms of execution-time. As a result, it is assumed that the retrieval-latency of the two basic (vector-database) pipelines will be lower than the one introduced by reranking and by hybrid-retrieval with reranking. Consequently, the best-performing pipeline concerning answer-quality might not be the fastest pipeline concerning end-to-end latency.

- **RQ3: Which timing components dominate the execution cost of the implemented RAG pipelines?**

H3: Generation time is anticipated to be the largest component of the overall processing time for almost all pipelines, while retrieval time is expected to grow in importance as more pipelines are added (reranked) or use a combination of both dense and sparse retrieval methods. The amount of indexing time will depend upon how the pipeline has been optimized for either in memory retrieval, a vector database, tuned chunking, or hybrid retrieval.

- **RQ4: How does the same RAG workflow behave, in terms of execution time, when moved from local to HPC deployment?**

H4: The major differences are expected to occur with respect to timing behaviour when using the same six RAG pipeline structures and the same five scenarios on both local and HPC systems. On an HPC system, it is expected that the overall time of a computation will be reduced significantly compared to a local environment; particularly where computations rely heavily upon data generation. However, there is no reason why all retrieval times would need to show improvement since vector database access, storage behaviour and service interactions can create overhead for short retrieval operations.

These hypotheses are designed to be compared and measured. They can

be evaluated using retrieval-quality metrics, answer-quality rubrics, timing data, and labels showing whether the workflow was run locally or on HPC.

1.6 Research Methodology Overview

The methodology includes a literature study, a software implementation, controlled experiments, and an analysis of the qualitative output from these studies. Figure 1.1 summarizes this workflow.

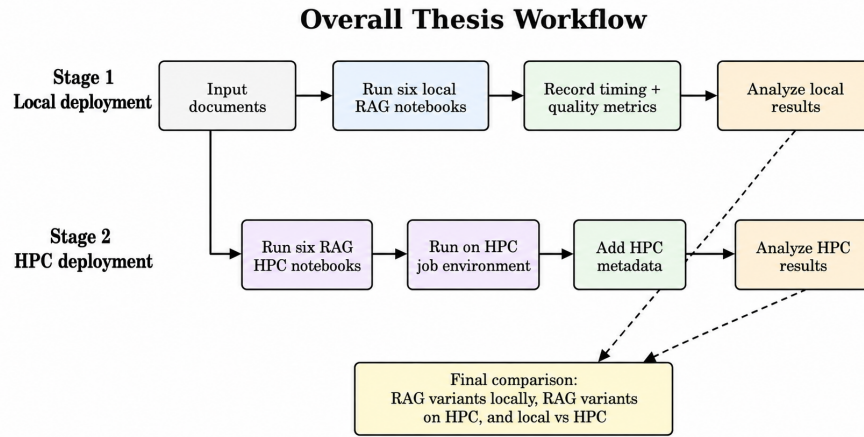


Figure 1.1: Overall thesis workflow showing the local deployment stage, the later HPC deployment stage, and the final comparison between RAG variants and execution environments.

The first step develops the theoretical and technological basis for the studies. It reviews LLMs, transformer architectures, how prompts are generated, RAG architectures, retrieval technologies, reranking, hybrid retrieval and HPC as a technology for deploying systems. Step one (i) provides the vocabulary and context necessary to understand both the implementation (ii) and the results (iii).

Step two (ii) represents the implementation of the research pipeline; to organize the implementation into a series of six Jupyter notebooks. The content of each notebook includes; loading documents, preparing chunks, creating or connecting to the appropriate store (vector database), running query scenarios, generating answers and saving results.

The third step (iii) represents the evaluation process for each of the pipelines. To evaluate each pipeline, timing information is collected based on seconds. Indexing time, retrieval time and generation time are all included in this measurement. Additionally to collecting qualitative measurements of how

well each pipeline performed, the source documents retrieved and the generated answers are also manually scored and analyzed utilizing deterministic metrics. The manual scores represent if the generated answer is correct or incorrect as determined by the evidence within the document corpus. Deterministic metrics utilized include retrieval hit, retrieval precision, retrieval recall, retrieval F1 score, ROUGE-L, BERTScore, semantic similarity and length of answer. Collecting both qualitative and quantitative measurements allows for an assessment of the trade-offs referenced in chapter 6.

1.7 Expected Contributions

1. **A reproducible local and HPC workflow for evaluating RAG pipelines.**

The thesis presents a uniform execution environment to test the RAG pipelines from the same document set, with the same evaluation questions, the same log formats, and the same evaluation workflow.

2. **An empirical comparison of six retrieval-level RAG variants.**

In this study, six retrieval-level RAG variants are compared: in-memory retrieval, Milvus vector-database retrieval, tuned Milvus vector-database retrieval, dense retrieval with reranking, hybrid retrieval, and hybrid retrieval with reranking.

3. **An evaluation of quality and efficiency trade-offs.**

This thesis evaluates the implemented pipelines through measurement of timing metrics, quality metrics for retrieval, and quality metrics for answers.

Future directions include investigating larger-scale system-wide alternatives (e.g., caching mechanisms, batch processing capabilities, ability to execute multiple processes concurrently, measuring throughput of individual components, memory usage monitoring), additional datasets, and larger model families.

1.8 Structure of the thesis

The structure of the remainder of the thesis is described below.

Chapter 2 provides an overview of the theoretical foundations necessary for the development of the thesis. The chapter addresses; Large Language Models (LLMs), the Transformer architecture, model adaptation approaches, prompt engineering, Retrieval-Augmented Generation (RAG), retrieval techniques, evaluation metrics and High-Performance Computing (HPC) concepts.

Chapter 3 provides a review of existing literature referring to LLMs deployment and inference optimization, RAG architectures and systems, retrieval techniques, and benchmarking methodologies.

Chapter 4 explains the methodology adopted. This includes a description of the research framework, the input data, the system architecture, the implemented RAG pipelines, experimental parameters, performance metrics, answer-quality measures, and reproducibility procedures.

Chapter 5 describes how the experiments are set up and executed. This includes descriptions of the software environment, models/services, input corpus, notebook execution sequence, local and HPC deployments, experiment scenarios, and result logging processes.

Chapter 6 reports and discusses the findings. The chapter will report and compare the results from each implemented pipeline by comparing them with respect to time-based metrics (timing); and retrieval/answer quality based metrics (retrieval quality and answer quality), as well as analyze the trade-off relationships between speed, retrieval quality, answer quality, and reproducibility.

Chapter 7 provides an overall summary of the conclusions drawn throughout the thesis, but also its limitations, and potential avenues for further research in areas such as additional datasets, larger model families, or other enhanced evaluation methodologies.

1.9 Summary

This Chapter introduced the motivations for this study, defined the research questions, provided an overview of the methodology used in this research, outlined what is intended as a contribution of this research, described how this study is organized, and identified the primary focus of this study which is to compare RAG pipeline approaches at the retrieval level, provide reproducible local and HPC execution workflow solutions, and to explore the trade-offs

among retrieval quality, answer quality, performance and reproducibility. The next chapter will describe the theory necessary to develop a better understanding of LLMs, RAGs, various retrieval methods, evaluation metrics and HPC deployment concepts.

Chapter 2 – Background and Theoretical Foundations

2.1 Introduction

This chapter establishes the necessary theory for developing and analyzing the RAG pipelines that were developed in this thesis. While it will establish some of the underlying theory for the Large Language Models (LLMs), it will focus primarily on those aspects of the theories that are specifically relevant to retrieval, inference time behaviour and differences between High-Performance Computing (HPC) and local deployment.

After providing a general introduction to LLMs and their use of the Transformer architecture, the chapter is intended to clearly distinguish RAG as a paradigm for developing large-scale Natural Language Processing (NLP) applications from several other paradigms which include prompt engineering, fine-tuning and training from scratch. After establishing the primary RAG application flow, the chapter covers the different methods for retrieval, reranking, hybrid retrieval and HPC concepts relative to this project. Finally, the chapter defines four categories of metrics that will be discussed later in the experimental evaluation: (i) System Performance; (ii) Retrieval Quality; (iii) Answer Quality; (iv) Local vs. HPC Timing Comparison.

2.2 Large Language Models (LLMs)

2.2.1 Overview

LLMs are deep learning-based language models which enable processing and generating natural language. Generally, these models are trained on massive datasets of text data and statistically learn relationships in natural language, including word, phrase, concept, and context relationships. Due to the nature of their training, LLMs can be utilized for various applications, including QA, document summarization, text generation, and extracting information from documents without explicit task-oriented programming.

Although, the effectiveness of using LLMs for real-world applications is directly influenced by the model itself; however, there are numerous factors that influence the effectiveness of the model within a larger system architecture. When utilizing LLMs, each time an inference operation is requested, the user provides an input query or prompt. After receiving the input prompt, the model processes the prompt layer-by-layer (many times) through a series of transformer layers and generates output tokens one at a time. While, not all operations require significant computational resources, operations involving longer prompts or multiple simultaneous requests may consume considerable resources. Consequently, there are several system-level considerations that can have a substantial impact on the performance experienced by users of LLM-based systems. These include prompt length, retrieval design, model serving behaviour, hardware selection, and the way repeated computations are handled.

This research focuses on utilizing an LLM as part of a RAG system. Therefore, while the primary experimental focus will not involve improving the weights within the LLMs; rather, it involves examining how different approaches to retrieving additional contextual information and overall system configurations affect both the quality and efficiency of the final generated answer.

2.2.2 Transformer Architecture

Modern LLMs are built upon the Transformer architecture introduced by Vaswani *et al.* [2]. The transformer architecture may be viewed as a sequential processing model, where a sequence of tokens is read into the model, and the representation of each token is iteratively updated based on the representations of the surrounding tokens. This is significantly different from earlier recurrent models that were primarily sequential in their operation, and therefore lacked much inherent or explicit parallelism.

The Diagram 2.1 shows the main building blocks used to transform input tokens into contextual representations and generated output tokens.

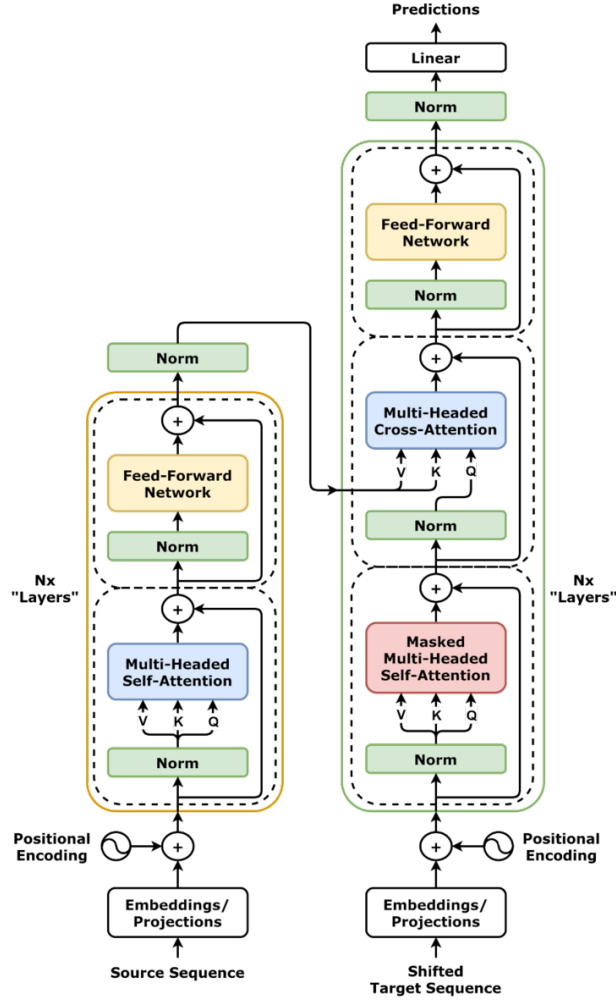


Figure 2.1: Transformer encoder-decoder architecture [2].

The most important idea in the Transformer is *self-attention*. In simple terms, self-attention is the mechanism that helps the model decide which words in the input are important for understanding each other. For example, in the sentence “the server restarted because it ran out of memory”, the word “it” should be linked more strongly to “server” than to unrelated words. Self-attention allows the model to make these links automatically. Each token is compared with the other tokens in the sequence, and the model gives higher weight to tokens that provide useful context.

More technically, the model creates three learned representations for each token: a query (Q), a key (K), and a value (V). The query represents what a token is looking for, the key represents what each token offers, and the value contains the information that will be passed forward if that token is considered relevant. The attention score is calculated as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V, \quad (2.1)$$

where d_k is the dimensionality of the key vectors. The softmax operation converts similarity scores into weights, and these weights determine how much information from each token is included in the updated representation.

The Transformer extends self-attention through *multi-head attention*. Instead of applying attention only once, the model uses several attention heads in parallel. One head may focus on grammatical structure, another may focus on entity relationships, and another may focus on long-distance dependencies. The outputs of the heads are then combined. This gives the model a richer representation of the input than a single attention calculation would provide.

The Figure 2.2 shows a Multi-head attention mechanism illustrating the linear projections of queries, keys, and values, followed by parallel attention heads and concatenation.

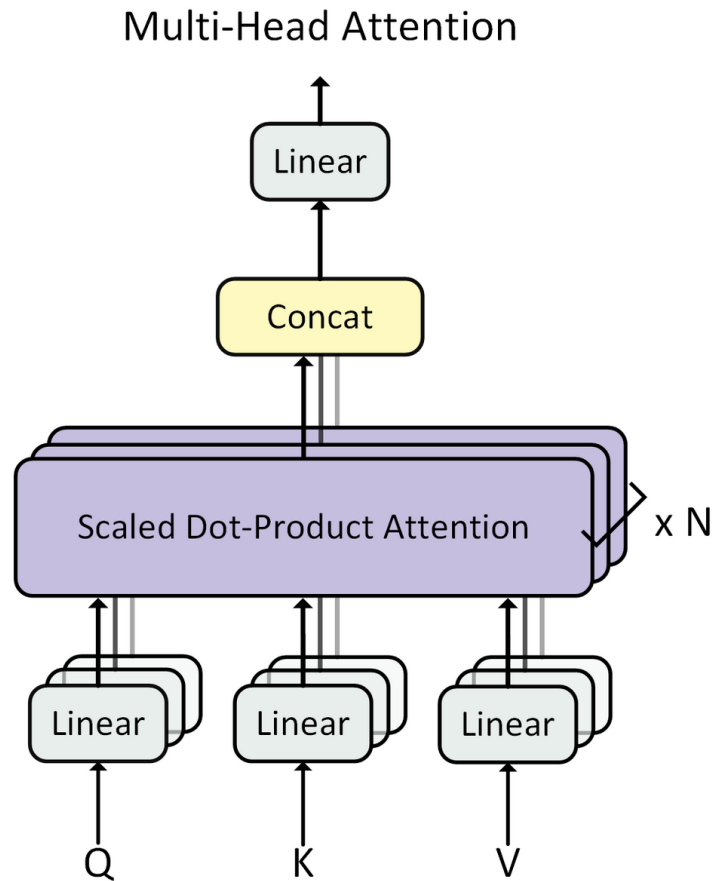


Figure 2.2: Multi-head attention mechanism [1].

The most important aspect of how the Transformer affects the way it behaves during runtime, is through its influence on generation. Typically, when using a model to generate something (such as an answer), the generation process is sequentially. That means that the model will produce one item

at a time. Therefore, long prompts, large amounts of data that have been retrieved, and/or multiple questions asked could potentially slow down the overall generation process. So, the RAG evaluation should consider, whether the generated answer was accurate, as well as the time taken to generate the answer, the time taken to retrieve the relevant information used by the model to generate the answer, and the total amount of time from start to finish (end-to-end).

2.2.3 Development Paradigms and their Effect on LLM Performance

There are a variety of methods to create LLMs-based applications. Each application development method may influence the behaviour of the model as well as that of the surrounding system. Prompting methods change the model input without changing model weights [11], while parameter-efficient methods such as Low-Rank Adaptation (LoRA) and adapter-based transfer learning update or add trainable parameters to adapt the model [12, 13]. This distinction is especially relevant to this thesis because the experiments were designed without optimizing model size; updating the model’s weight values; fine-tuning the model; quantization of the model nor pruning the model. Rather, this thesis focuses upon retrieving-level design choices made during RAG workflow construction, as well as comparing the performance of those RAG workflows within both local computing environments and HPC environments.

This distinction is important because retrieval-level techniques improve the information supplied to the model, while model-level techniques modify the model itself. The scope of this thesis requires distinguishing the difference between prompting, RAG, fine-tuning, and training from scratch, but the implementation work focuses on deployable RAG pipelines and their performance in local and HPC environments.

Table 2.1 provides an overview of the main LLM development paradigms and shows how each one relates to the scope of this thesis.

Paradigm	How it affects LLM performance	Role in this thesis
Prompt engineering	Changes the instructions and context given to a fixed model. It can improve answer style and task following, but it does not update the model’s knowledge.	Background only; prompts are kept controlled across pipeline comparisons.
RAG	Adds external retrieved context to the prompt. It can improve factual grounding and domain specificity without changing model weights.	Main focus; several retrieval variants are implemented and compared.
Fine-tuning	Updates some or all model parameters using task-specific data. It can improve task performance but requires additional training resources and model management.	Out of experimental scope; discussed only to distinguish it from RAG.
Training from scratch	Learns all model parameters from large datasets. It provides maximum control but requires very large compute, data, and engineering resources.	Out of scope; included only as background.

Table 2.1: LLM development paradigms and their relevance to this thesis.

2.3 Retrieval-Augmented Generation (RAG)

2.3.1 Concept and Motivation

RAG uses a combination of a language model and an external retriever. In this approach instead of only getting the answer from the language models’ own parameterized knowledge base it first identifies appropriate passages from a document collection, which are used as additional input to provide contextual support to the language model. Using the RAG method has been shown to improve factual grounding and decrease the number of unsupported answers produced by the model; especially when the query is specific to a particular domain or type of data that may be less frequently represented within the models’ parameterized knowledge base [5].

In this thesis, the use of RAG is driven by a real-world constraint; i.e. that it will be simpler to update a document set than to fine-tune or retrain a model each time there are changes in the data on which the model is trained. A RAG system can search for and retrieve information from project documentation, user manuals, policies or synthetic datasets as long as you simply update the search space (update the searchable document set) and do not alter the model weights. The added flexibility of this approach generates new design

issues relative to loading documents, splitting them into blocks, generating embeddings for the individual blocks, determining if additional ranking is needed, and inserting the found contextual information into the final prompt.

2.3.2 Architecture and Workflow

The RAG workflow includes two high-level stages; (i) An indexing stage and (ii) a query-time stage. During the indexing stage, the documents are loaded, cleaned (noise reduction), broken down into manageable pieces (chunks) of data and then embedded using an embedding model for use at query-time. The embedded chunks are then placed in a retrievable structure (e.g., in memory document store or vector database). At query-time, the user's question (prompt) is sent through the retrieval process where relevant pieces of data are selected and returned to the system. The selected data is then appended to the prompt and sent to the LLM which will generate the final answer.

The workflow can be summarized as follows:

1. **Preparation of Documents:** Source documents collected, cleaned and split into smaller “chunks” that can be fitted into model's context window.
2. **Indexing:** Chunks stored in retrieval structure. Dense systems store embeddings, sparse systems store lexical statistics. Hybrid systems use both methods.
3. **Processing of Queries:** query from user converted to representation needed by retriever.
4. **Retrieval:** top candidate chunks returned based on lexical similarity, embedding similarity or combination of both.
5. **[Optional] Re-ranking:** reranker filters (re-ranks) initial candidates so most relevant chunks will appear first.
6. **Construction of Context:** selected chunks inserted into prompt with original question.
7. **Answer Generation:** LLM generates answer using question and retrieved context.

8. **Evaluation and Logging:** system logs timing, quality of retrieval, output-quality metrics for comparison.

High-level view of pipeline is important because each implemented pipeline only changes some parts of workflow. For example baseline-pipeline uses simpler retrieval method than other pipelines. Vector-database pipeline changes storage mechanism, reranker pipeline adds second ranking step, hybrid pipelines combine sparse and dense retrieval signals.

2.3.3 Retrieval Techniques

When selecting the best possible retrieval quality for RAG, there will always be some limitations to how well the retriever can perform its job. When the retriever finds very little useful context, even though the generator has the capability to create a correct answer (if the language model itself is adequate), then the generator will likely have no choice but to create a weakly-supported, possibly incomplete answer. The retrieval techniques examined during this research were dense retrieval, vector-database retrieval, reranking, and hybrid sparse-dense retrieval. As a supporting concept, sparse retrieval was included here due to it being a component within the hybrid retrieval design.

The mentioned methods were selected because they provide the opportunity to alter the retrieval choices of an already developed Large Language Model (LLM), without having to make changes to the model. Dense retrieval represents the most basic level of retrieving based on semantics, vector database retrieval represents the ability to store embeddings for extended periods of time, reranking represents whether or not a secondary relevance model can improve ordering in the candidates, and Hybrid Retrieval represents if lexical information could be used in addition to semantic information. The other models (model fine-tuning, etc.) are being left out of this evaluation, since they would require altering the model instead of simply changing how it retrieves.

1. **Sparse Retrieval:** Employs lexical signals, such as exact terms and term frequencies, to align queries with documents. Best Matching 25 (BM25) is a prevalent algorithm in this domain, effective when queries and documents share key terms. However, its reliance on exact matches can slow down retrieval of sections with similar ideas expressed differently. Although not tested independently in

this study, BM25 representations were incorporated into the hybrid-retrieval pipelines [14].

2. **Dense Retrieval:** Dense retrieval uses high-dimensional embeddings for queries and documents, focusing on semantic similarity rather than exact wording. This approach allows for the identification of semantically equivalent concepts, exemplified by the Dense Passage Retrieval (DPR) algorithm. In the study, dense retrieval employs embedding based searches at in-memory or Milvus-backed document repositories [15].
3. **Vector Database Retrieval:** Vector databases enable the storage of embeddings and performing nearest neighbor queries on extensive collections of documents, making them particularly beneficial for large, persistent document repositories. This study employs Milvus for the vector database pipeline, specifically the Milvus-backed version of the dense retrieval variant. This choice is consistent with approximate nearest-neighbor indexing approaches used for scalable vector search [16, 17].
4. **Hybrid Retrieval:** Hybrid retrieval integrates sparse and dense features for enhanced performance. Sparse retrieval specializes in exact terminology, while dense retrieval is adept at recognizing semantic similarities. This combination improves recall and robustness across different query types by combining BM25-style lexical matching with embedding-based semantic matching [14, 15]. The study utilizes Milvus’s hybrid retrieval function for implementation.
5. **Reranking:** Reranking is a subsequent stage of information retrieval where an initial candidate list of documents, produced by a retriever, is scored for relevance. Although cross-encoder rerankers offer greater accuracy compared to basic vector similarity, they introduce extra computational demands. This study applies reranking to both dense and hybrid retrieval methods [18, 19].

A major trade-off associated with developing a system for performing retrieval is that stronger retrieval systems typically require longer times to execute. On the other hand, although hybrid retrieval and reranking may provide increased flexibility in retrieving contextual information by improving context selection relative to simpler retrieval systems, these approaches

result in slower execution times. This trade-off represents one of the primary measures of success developed during this research.

2.4 High-Performance Computing (HPC) Systems

2.4.1 Overview

HPC systems are collections of interconnected computers (usually referred to as nodes) which have been designed to support computationally intensive workloads requiring greater computational power, memory, and/or data storage throughput than a single desktop computer can provide. For this project, HPC is important because the same RAG workflow used locally is deployed in a managed cluster environment where it can be tested under different resource conditions.

In contrast to a typical desktop workstation, HPC systems do not typically execute the users' requests immediately. Instead, when a user logs into an HPC system they will use a login node to create their own directories, prepare their code and job submission scripts and then submit their job to a scheduler (such as Slurm) [10]. At this point, the scheduler determines what resources (compute nodes, Central Processing Units (CPUs), Graphics Processing Units (GPUs), etc.) should be allocated to each submitted job based upon predetermined policies of the system. Although HPCs are extremely useful for conducting reproducible batch experiments; the process of doing so introduces operational considerations (e.g., queue time, startup overhead associated with submitting a job, accessing storage, creating containers).

2.4.2 Architecture and Components of HPC

The HPC system consists of multiple parts:

- **Local workstation:** The user's personal computing device, used to write code, connect to the cluster, transfer files, and review results.
- **Login node:** The gateway into the cluster. This is where files are edited, environments are prepared, and jobs are submitted. Heavy experimental workloads should not be executed on the login node.

- **Scheduler/resource manager:** The component that receives job submissions, queues them, assigns resources, and executes jobs on compute nodes. Slurm is an example of such a scheduler [10].
- **Compute nodes:** The nodes on which the actual workload is executed. Some compute nodes are CPU-only, while others include one or more GPUs or other hardware.
- **Shared storage:** The storage system used for datasets, notebooks, vector indexes, logs, and result files. Shared storage can affect RAG workflow performance because document collections and indexes may be read repeatedly.
- **Software environments:** HPC clusters provide mechanisms for reproducing software environments, including module systems, virtual environments, and, in some systems, containers such as Apptainer (container platform).

2.4.3 Parallelism, Caching, and Batching in RAG Workloads

The concept of parallelism is largely defined by how many different systems execute simultaneously versus how many models train simultaneously. Examples of key types of parallelism include independent query parallelism, batch evaluations, parallel document processing and parallel embedding generation. An example would be dividing up a series of test questions among different worker nodes/submitting each question as its own job. Likewise, a large document could be broken into smaller pieces (chunks) and embedded in batches; not piece-by-piece.

Similar to parallelism, caching is critical for RAG pipelines to eliminate redundant computation. When using the same documents across a series of experiments it is likely to find that the embeddings and indexes can be reused. Furthermore, evaluating the same query set on numerous occasions, can cache repeated results for debugging purposes or for comparing the performance of similar systems. While caching can significantly increase speed, care must be taken to ensure that all cached and non-cached results are properly accounted for when making comparisons.

In this thesis, these factors are treated as deployment considerations rather than as separate experimental variables. The implemented comparison fo-

cuses on the timing fields recorded by the notebooks.

2.4.4 Job Scheduling and Resource Management

HPC execution is dependent on job scripts that specify how much HPCs resource each experiment requires (for example, how many nodes, CPUs, GPUs, etc., memory, what is the runtime, where does the output go). Then, the scheduler puts the job into a queue, and starts the job when there is an opening for resources.

Job scheduling is important for the HPC phase of this thesis because it can impact both the performance and reproducibility of the experiments. If measuring end-to-end wall clock time from outside the application, it also captures all of the time spent waiting in line with the scheduler; whereas, application run time measures only the time that the application spends running once started. For this reason, it is important to conceptually separate queue time and service-startup overhead from application-level timing. The timing fields noted in the notebooks indexing time, retrieval time, generation time, and overall runtime, are the main focus of the implementation

2.4.5 Containerization

Containerization enables reproducibility by packaging software dependencies into a portable environment. The local experiment uses Python virtual environments, Docker, Ollama and Milvus, while the HPC experiment uses the cluster-approved serving and execution workflow. The goal is to keep the same project files, models, datasets and logging format across all environments and adapt only the system setup required by the cluster to run on it.

For RAG workloads, reproducibility relies on consistent Python packages, model versions, embedding model versions, vector database configuration, query sets and result logging. Containerization reduces the amount of environment drift between the local and HPC stage.

2.4.6 Performance Optimization

Several system-wide factors may impact the RAG workflow run time when utilizing an HPC environment. The issues that are mentioned bellow are

considered in the context of a discussion regarding HPC versus local execution in this thesis. The experimental comparison was based upon the timing fields recorded from the notebooks utilized to execute the experiment.

The most influential factors on performance include:

- **Index Loading/Storage Access:** Indexes and/or collections of documents may have a high cost associated with them when being loaded into memory from shared storage. This can significantly reduce the need to repeatedly pay this overhead by copying data that is accessed frequently to local storage on each node, or by maintaining indexes that remain "warm".
- **Embedding/Generation Parallelism:** Due to the fact that many operations are independent, document chunking, embedding generation, and evaluation queries can often be executed in parallel.
- **Batch Size:** Batching larger quantities of documents can improve the throughput for either generating or retrieving embeddings; however, it may also increase the amount of memory required per batch, which can result in longer wait times before a query will be processed.
- **Caching Policy:** By reusing cached versions of embeddings, indexes, and repeated retrieval results, unnecessary computations can be avoided. If caching is used, it should be documented so that repeated runs are interpreted correctly.
- **Resource Allocation:** Selecting the correct combination of CPU, GPU, memory and wall-time requests for an experiment will affect how long an experiment runs, where the scheduling algorithm places the experiment within the cluster, and potentially the overall performance of the experiment.

These considerations are directly related to the broad deployment goal of the thesis. The experiments compare retrieval-level methods (dense retrieval, vector database retrieval, hybrid search, reranking), while system-level strategies such as caching, batching, and parallel execution describe how the same workflow is deployed and interpreted on HPC.

2.5 Evaluation Metrics in Model Deployment

Evaluating an RAG pipeline requires more than simply measuring how fast it is. An "incredibly" fast system will be completely useless if it retrieves a lot of irrelevant documents or provides no supporting evidence for the generated answers. In order to measure these aspects of the evaluation of an RAG system, this thesis uses three categories of metrics: (1) system performance; (2) retrieval quality; (3) answer quality. The first category includes indexing time, retrieval time, and generation time. The second category includes retrieval hit, precision, recall, and F1. Answer quality is evaluated by combining manual correctness scores with ROUGE-L, BERTScore, sentence embedding semantic similarity and answer length [20–22]. The classification as well as the categorization of the used evaluation metrics corresponds to the requirement to measure the behaviour of machine learning systems and the quality of their generated language [23, 24]. While this thesis did not create an implementation of the Retrieval Augmented Generation Assessment (RAGAS) framework, the evaluation of the thesis follows the same general categorization that was developed by the authors of the RAGAS framework into a separation between retrieval behaviour, generated-answer quality, and execution costs [25].

2.5.1 System Performance Metrics

System performance metrics measure the operational efficiency of the pipeline. They are included because retrieval improvements are only useful if their quality benefits can be compared with their execution cost. Performance metrics track the efficiency of each processing element (chunker, indexer, LLM) in the overall pipeline process:

- **Indexing time:** The amount of time required to load and index the documents; chunk them into smaller pieces; generate an embedding from each piece; and/or create, build, and populate the document store.
- **Retrieval time:** The amount of time it takes to find all candidate passages for a query. For the reranked pipelines, the retrieval time will be longer due to the fact that reranking occurs during the retrieval portion of the pipeline within the provided notebooks.
- **Generation time:** The amount of time required by the LLM to produce answers once the prompt has been created.

The above metrics are logged through the three fields `indexing_time_s`, `retrieval_time_s`, and `generation_time_s`. Broader metrics such as end-to-end latency, throughput, CPU/GPU utilization, memory usage, and storage usage are relevant to HPC deployment analysis, but the current evaluation framework keeps the core comparison focused on the timing fields shared across the implemented runs.

2.5.2 Retrieval Quality Metrics

Retrieval quality metrics measure if the retriever finds the right evidence before it generates an answer. These are especially important for evaluating dense retrieval, vector database retrieval, hybrid search, reranking and hybrid search with reranking.

- **Hit:** Does at least one expected supporting document appear among the retrieved sources?
- **Precision:** What fraction of retrieved documents match manually identified supporting documents?
- **Recall:** What fraction of expected supporting documents appears among the retrieved sources?
- **F1:** Harmonic average of precision and recall.

2.5.3 Answer Quality Metrics

The quality metrics of the answers determine how well LLM's generated output will satisfy the requirements to provide a high-quality response. Since there is no requirement that an LLM respond within a certain amount of time, these metrics assess the quality of the LLM's output itself rather than just its execution time.

- **Manually-scored:** An LLM's answer is manually scored (correct=1.0; partially correct = 0.5; incorrect = 0.0) based on the ground-truth answer given for it.
- **Manually-labeled:** A category is assigned to each answer: "correct", "partially correct" or "incorrect".

- **Manually-noted:** Comments provided based on evidence explain why an answer was deemed "missing information"; "contains incorrect statements"; or "lacks supporting reasoning".
- **ROUGE-L F1:** A lexical-overlap-based score (using ROUGE-L) compares the generated answer against the ground-truth answer [20].
- **BERTScore F1:** Semantic-similarity score calculated from contextualized BERT word embeddings [21].
- **Semantic-similarity using sentence embeddings:** Cosine-similarity between the generated answer and the ground-truth answer, both represented using sentence embeddings [22].
- **Length of answer:** Number of words in the generated answer - a supportive descriptive metric.

As such, this assessment uses both manual judgments and deterministic metrics. Although the manual notes may include some discussion about grounded-ness, completeness, relevance, and supported claims that could impact correctness, none of those dimensions are currently being recorded separately as numeric fields in the existing evaluation script.

The Table 2.2 illustrates the metric used on this thesis as well as the purpose of each.

Metric group	Implemented metrics	Purpose
System performance	Indexing time, retrieval time, generation time	Shows how expensive each pipeline is to run in the local setup.
Retrieval quality	Retrieval hit, retrieval precision, retrieval recall, retrieval F1	Shows whether the retriever finds the expected supporting documents.
Answer quality	Manual score, manual label, manual notes, ROUGE-L F1, BERTScore F1, semantic similarity, answer length	Shows whether the final answer is correct and similar to the ground-truth answer.
Deployment comparison	Local execution results; HPC metadata such as environment, hardware, scheduler, container, and storage details	Shows how the same workflow is compared between local execution and HPC.

Table 2.2: Metric groups used in the thesis evaluation framework.

2.5.4 Local versus HPC Comparison Metrics

When the same RAG workflows are executed locally and on HPC, the comparison should separate pipeline behaviour from environment-specific behaviour. If the same data, models and parameters are used for retrieval accuracy and answer quality, they should be generally comparable. However, performance metrics can vary due to how resources are allocated (e.g., scheduling, etc.), how quickly data can be retrieved or stored, as well as scheduler overhead, batching and parallelism in the execution of the code.

To ensure a baseline comparison, the following performance data must be captured: execution environment; hardware description; number of workers executing; batch size executed; cache status; index creation time; retrieval time; generation time; and overall runtime. The goal here would be to allow for a direct comparison of the same pipeline run locally and on an HPC system without having to consider differences in the experimental design.

2.6 Summary

In this chapter, the theoretical basis for this thesis was presented. It introduced Large Language Models (LLMs), the transformer architecture and a clear difference between model level techniques like fine-tuning and retrieval-level techniques like Retrieval-Augmented Generation (RAG). The major components of the RAG workflow are then presented and include document preparation, indexing, retrieval, optional reranking, prompt construction, answer generation and evaluation.

Also included in the chapter were the methods of retrieval that are being employed in the two different implemented pipeline architectures; dense retrieval, vector database retrieval, hybrid sparse-dense retrieval, and reranking. The High-Performance Computing (HPC) aspects relating to both the local versus HPC comparisons were discussed along with defining the metric groups utilized in the evaluations which consist of three categories of metrics: system performance metrics, retrieval quality metrics, answer quality metrics and deployment timing comparison metrics.

Chapter 3 – Literature Review

3.1 Introduction

This literature review aims to compile prior research on LLMs, inference deployment, RAG-based architectures and evaluation methods related to this thesis. The increasing scale and complexity of LLMs are requiring ever greater amounts of memory, computation, latency, and operational costs when they are deployed. In contrast with these challenges, prior research on RAG has demonstrated how external retrieval may enhance the groundability of the responses generated by models through linking the output of the model to a document repository.

As such, this chapter synthesizes prior research into three main categories: (i) approaches for deploying LLMs locally, cloud-based or using High-Performance Computing (HPC) infrastructures; (ii) optimization strategies for inference, which impact the scalability and memory utilization of models; (iii) RAG architectures that include document indexing, retrieval and response generation. Additionally, it reviews benchmarks and evaluation methodologies commonly employed to evaluate both the quality of models and the quality of retrievals as well as the overall performance of systems.

It is not in the intention of this review to discuss each optimization methodology at length, but rather to provide context for this thesis. Specifically, this thesis will compare six different RAG pipeline configurations at the level of retrieval with respect to timing behaviour over both local and HPC execution.

3.2 LLM Deployment Approaches

3.2.1 Local and Cloud-Based Deployments

Research into the deployment of LLMs often compares cloud-based infrastructure, local systems, and managed compute environments. The main differences between these types of deployments are related to their available resources (such as CPU/GPU), how scalable they can be, the total cost of ownership, and the performance characteristics of each environment.

- **Deployments using cloud-based infrastructure** utilize the elastic computing resources provided by major cloud vendors such as Amazon Web Services (AWS), Microsoft Azure, and Google Cloud. These services support the use of distributed inference engines, highly scalable GPUs, and also support the ability to manage model serving applications at scale, thus allowing for reliable execution of very large LLM workloads. Additionally studies have shown that through automated scaling, and distributed storage a high degree of efficiency is possible when executing multi-billion parameter models [26]. Some negative aspects associated with deploying models in the cloud include: the high operating expense (e.g. hourly charges for running the service), data-transfer fees, vendor lock-in, and network latency (the time it takes for data to travel over the network).
- **Deployments using local infrastructure** require a user to deploy their models to a workstation-class GPU or an on-premises server. Local deployments offer several advantages including: predictable performance, low latency per user, and strong data governance. A significant disadvantage of local deployments is the inability to scale up and down to accommodate multiple users or meet the growing demand for increasingly larger LLM models.

Given the current trade-offs between control (local deployments) and scalability (cloud deployments), HPC-based deployments are relevant because they provide managed compute resources, scheduling, shared storage, and reproducible execution workflows that differ from both personal workstations and persistent cloud services.

3.2.2 HPC-Based Deployments

High-Performance Computing (HPC) systems offer a potential solution for scaling LLM workloads because they provide clustered compute resources, high-bandwidth connectivity, and large overall memory capacity. In contrast to cloud-based environments optimized for loosely connected service-based applications, HPC clusters allow for rapid communication between each node, enabling both efficient distributed training and inference [8].

A great deal of prior work focuses on distributed training frameworks (Megatron-LM [27], DeepSpeed’s ZeRO [28]) that utilize tensor, pipeline, and data parallelism to enable the training of extremely large models. These

approaches also support distributed inference for models whose memory requirements exceed the capacity of a single device.

In addition to the above references, other researchers have examined aspects of deep learning in HPC environments including high-speed interconnects such as InfiniBand and NVLink, the effects of parallel file system Input/Output (I/O) performance on LLM, and the impact of scheduler overhead on performance. These studies show that hardware, storage, and scheduling decisions can affect machine-learning workloads. For this thesis, those same categories are used to interpret retrieval-based workloads, where retrieval latency and storage throughput affect overall RAG performance.

Prior research has shown that system-level decisions, including containerization with Apptainer and workload scheduling with Slurm, can improve the efficiency and scalability of different Machine Learning workloads. This thesis applies the same type of systems-aware reasoning to the implemented LLM inference and RAG deployment workflow.

Although HPC platforms can support larger and more controlled workloads than a laptop, the relevant point for this thesis is the practical comparison between local execution and managed HPC execution of the same RAG workflow.

3.2.3 Model Serving Frameworks

A number of high-throughput model-serving frameworks have been developed to scale LLMs:

The **Virtual Large Language Model (vLLM)** framework introduced the PagedAttention mechanism to improve Key–Value (KV)-cache management and increase throughput under concurrent request workloads [29]. In this thesis, vLLM is relevant because it is used to serve the generation, embedding, and reranking components in the HPC workflow.

The **NVIDIA Triton Inference Server** is an inference server framework capable of running multiple frameworks concurrently using dynamic batch sizing and GPU-aware task scheduling. However, it was not initially designed with LLMs in mind; therefore, it has limited ability to optimize LLM-specific parameters.

The **Text Generation Inference (TGI)** framework was created for the purpose of optimizing performance when deploying transformer-based text generation models. TGI supports tensor-parallelized inference, quantization,

and continuous batch processing.

RayServe is a distributed orchestration tool for scaling applications across cluster resources. While RayServe does provide a flexible way to deploy applications across multiple machines, its primary function is focused on the distribution of tasks across nodes rather than providing optimizations for inference speed.

Therefore, while several serving frameworks can be used to distribute application logic, this thesis focuses on the practical role of vLLM-based serving in the implemented HPC workflow and on how deployment overheads are separated from retrieval and generation timing.

3.3 Optimization Techniques for LLM Inference

3.3.1 Quantization and Pruning

Quantization reduces memory usage and computational cost by converting model weights and activations from higher-precision formats such as Single-Precision Floating-Point (32-bit) (FP32) or Half-Precision Floating-Point (16-bit) (FP16) to lower-precision formats such as 8-bit Integer Precision (INT8) or 4-bit Integer Precision (INT4). Methods including LLM.int8 [30], Generalized Post-Training Quantization (GPTQ), and Activation-aware Weight Quantization (AWQ) allow for significant reductions in resource utilization with minimal losses in accuracy. This can enable larger batch sizes and faster inference.

Pruning is a method to remove redundant weights from an existing model to reduce the size of the model and computational complexity. Although early research focused primarily on vision models, recent work has applied pruning to Transformer models, and has demonstrated that structured sparsity can be used to reduce compute requirements without large losses in accuracy [31].

There are both advantages and disadvantages for using quantization and pruning in HPC environments. These techniques can reduce memory use and support larger batch sizes, but they may also introduce accuracy or compatibility trade-offs. In this thesis, they are treated as background techniques rather than implemented experimental variables.

3.3.2 Knowledge Distillation

Distillation has been shown to allow a larger "teacher" LLM to be reduced into a smaller "student". For example, models such as DistilBERT [32] have demonstrated that many capabilities of an LLM might be preserved in a model that is significantly cheaper than the original model.

The primary advantage of using knowledge distillation to train a "student" neural network (i.e., one that learns from a larger pre-trained teacher) is that it results in a significant reduction in both latency and memory requirements. Distillation has no direct impact on knowledge grounding; therefore, improving retrieval accuracy, or reducing hallucinations are not addressed. As such, knowledge distillation was not considered an experimental variable for this study; rather, was viewed through the lens of the retrieval system's design and deployment.

3.3.3 Parallelization and Sharding

Strategies to enable Data Parallelism, Tensor Parallelism and Pipeline Parallelism, represent the most important ways to scale LLMs. Tensor-Parallelism and Pipeline-Parallelism as used in Megatron-LM [27] are key to enabling distributed execution. The use of parameter, gradient and optimizer state shards in DeepSpeed ZeRO [28] and PyTorch Fully Sharded Data Parallel (FSDP) can be seen as an extension of the ideas behind those two forms of parallelization.

Although, there is a substantial body of work analyzing the impact of these methods on the training process, this thesis treats parallelization and sharding mainly as deployment background. The implemented comparison focuses on the measurable behaviour of the RAG workflow rather than on redesigning the language model execution engine.

3.3.4 Caching and Memory Optimization

FlashAttention is an attention optimization method that reduces memory movement and improves the efficiency of attention computation without changing the mathematical result of attention [33].

The PagedAttention mechanism used by vLLM splits the Key-Value Cache (KV-cache) into paged blocks, which improves performance in systems that

serve multiple requests at once [29].

In addition to sparsification, eviction strategies and cache compressions, researchers are actively exploring other optimizations. However, most of them are still in an experimental phase.

Due to multi-user workloads and job schedulers, HPC environments have varying amounts of available memory. For this thesis, caching and memory optimization are relevant because they influence model-serving behaviour and repeated execution costs, even though they are not the main experimental variables.

3.4 RAG Systems and Optimization Strategies

3.4.1 Existing Frameworks

The major frameworks for building RAG pipelines (LangChain [34], LlamaIndex [35], and Haystack) offer modules to build the pipeline. Their common role is to organize document loading, embedding, retrieval, prompt construction, and answer generation. This thesis uses this framework-oriented view to structure the implemented pipelines and keep retrieval choices comparable.

3.4.2 Retrieval Efficiency and Hybrid Approaches

The combination of sparse (like BM25) and dense (like DPR) retrieval techniques may be used in order to obtain a better recall rate than each single technique could offer on its own [14, 15]. Dense retrieval techniques have shown good results regarding query-document similarities, while sparse techniques show high quality for retrieving all the exact keywords mentioned in a query.

The evaluation of the hybrid retrieval strategy as a retrieval level variant has been carried out within the scope of this thesis. Its performance will be discussed together with timing behaviour since the use of both sparse and dense retrieval strategies is able to increase the number of retrieved relevant documents; however, they can also lead to an increased cost in terms of time needed by the system for finding them.

3.4.3 RAG on HPC Environments

RAG systems are often discussed in cloud computing or enterprise deployment settings, where services may exist persistently, dynamic auto-scaling may be available, and container orchestration may be part of the infrastructure.

In contrast, HPC systems utilize batch scheduling (e.g., Slurm) and employ parallel storage systems (e.g. Lustre, GPFS). The jobs executed on these systems do not remain active after their completion. These differences create additional complexities regarding how to serve, retrieve, and perform distributed inference [8–10, 36].

3.5 Benchmarking and Evaluation Studies

Benchmarking methods such as MMLU, HELM [24] and the OpenLLM leaderboard evaluate LLM performance. The MLPerf benchmark [37], however, is designed to measure inference performance. As such, these benchmarks represent a good reference point for evaluating LLM performance; they do not, however, provide an adequate framework for the workflow level evaluations presented in this thesis.

As indicated above, evaluation of RAG systems requires consideration of two key aspects of system performance. These include the overall quality of the retrieved answers (i.e., answer quality), and the quality of the actual retrieval process that generates those answers (i.e., retrieval quality). Specific benchmarks for RAG systems, such as KILT, evaluate the retrieval performance. In contrast, other evaluation frameworks focus on assessing the correctness, faithfulness, and relevance of the generated answers. The thesis will use a workflow-level evaluation method, which will compare the timing of the six implemented RAG pipelines along with their respective answer quality and retrieval quality metrics, for the local implementation only.

Therefore, the evaluation provided in this thesis is different than general-purpose benchmark suites. Rather than ranking numerous models using many publicly available tasks, it evaluates a small controlled number of RAG configuration options on the exact same corpus, under identical scenario-based conditions, and when compared locally versus High-Performance Computing (HPC) based timing conditions.

3.6 Positioning of This Thesis

The reviewed literature motivates several design choices in this thesis:

- LLM deployment studies highlight the importance of hardware, serving frameworks, and resource management.
- RAG studies highlight the importance of retrieval quality, grounding, and answer quality.
- HPC deployment practices highlight the importance of scheduling, shared storage, containers, and reproducibility.
- This thesis combines these perspectives in a controlled comparison of six retrieval-level RAG variants across a local and HPC workflow.

The contribution is therefore practical and bounded: the thesis does not claim to replace broad benchmark suites or exhaust the literature, but it applies established concepts to a reproducible implementation and evaluates how retrieval choices affect timing, retrieval quality, and answer quality.

Table 3.1 summarizes how each research area informs the scope and design of this thesis.

3.7 Summary

This chapter reviewed the literature on LLM deployment strategies, inference optimization techniques, RAG system design, and evaluation methods for LLMs and RAG. The reviewed work shows that deployment environment, model-serving approach, retrieval design, and evaluation metrics all influence the behaviour of practical RAG systems.

Therefore, chapter 4 uses these concepts to define the methodology for comparing the implemented RAG pipelines, their deployment workflow, and the metrics used to evaluate timing, retrieval quality, and answer quality.

Area	Main contribution from literature	Limitation for this thesis scope	Use in this thesis
LLM deployment	Explains trade-offs between local, cloud, and managed compute environments.	Does not by itself evaluate retrieval behaviour or answer grounding.	Defines why local and HPC execution should be compared separately.
Inference optimization	Describes serving, quantization, pruning, caching, and parallel execution strategies.	Many techniques change model execution rather than retrieval design.	Used as background while keeping the experiment focused on retrieval-level variants.
RAG systems	Shows how retrieval can ground generated answers in external documents.	General RAG descriptions do not determine which retrieval variant is best for this corpus.	Motivates the comparison of dense, vector-database, reranking, hybrid, and hybrid-reranking pipelines.
Benchmarking and evaluation	Provides established quality and performance measurement categories.	General benchmarks do not directly match the project corpus, scenarios, and logging workflow.	Supports the selected timing, retrieval-quality, and answer-quality metrics.

Table 3.1: Summary of related work areas and their relevance to this thesis.

Chapter 4 – Methodology

4.1 Introduction

The purpose of this chapter is to describe how the RAG pipelines were built, tested, and assessed in this thesis. It will show how the theory on LLMs; semantic search, reranking, hybrid retrieval models, HPC deployment, relate to the proposed experimental notebook-based workflow.

The methodology used a controlled comparative approach. First, a baseline RAG pipeline is created and subsequent pipeline versions are generated by adding different retrieval-levels. In order to allow for the attribution of differences in time, retrieval quality, and answer quality; mainly to the design choices made during retrieval and ranking, each version of the pipeline uses the same document set, query scenario sets, logging framework, and assessment process.

4.2 Research Framework and Design

The approach compares six RAG pipeline variants. The implemented variants include a basic in-memory pipeline, a Milvus vector-database pipeline, a tuned vector-database pipeline, a reranking pipeline, a hybrid retrieval pipeline, and a hybrid retrieval with reranking pipeline. The main focus is to evaluate how these choices affect retrieval quality, answer quality, and execution time. The result logs identify these variants as `basic`, `vectordb`, `vectordb_tuned`, `reranker`, `hybrid`, and `hybrid_rerank`.

The comparisons were made by using a "local first" process. Each of the notebooks was initially tested on a local computer to evaluate the functionality of all code, services, prompts, results logging, and all evaluation script functions. The same workflow was executed in HPC mode with changes to the model service and job execution to account for differences in environments. The only difference from the local environment in terms of how it executes was that the models were created on Slurm (an open source resource manager) and served as vLLM (a cloud-based LLM model service), whereas the notebooks used the same pipeline structure and logging process.

By focusing on retrieving and ranking, and keeping the generator model consistent across experiments wherever possible, the comparison isolates the effects of changing retrieval components, including dense retrieval, persistent vector storage, tuned chunking, reranking, sparse-dense hybrid retrieval and hybrid retrieval with reranking.

4.3 Dataset and Input Documents

The experiments use the OrchestraAI synthetic enterprise dataset developed specifically for this thesis in a local project directory. This dataset includes 54 .docx files, which are located within ten different departmental folders (Customer Support, Engineering, Finance, Human Resources/HR, Information Technology /IT, Logistics, Product, Sales, Security and Q&A Handbooks). In total the documents contain company policy and descriptions of policies; customer service procedures; on-boarding work flow; procurement processes; product development descriptions; as well as cross functional operational descriptions.

Each notebook uses the Haystack `DOCXToDocument` converter to load the same document set recursively. While documents are loaded, they are enriched with additional metadata – primarily the original source filename and department. This metadata is used in manually evaluating retrieved documents when the retrieved documents are saved to the `results.csv` for later comparison purposes.

The different chunking configurations are unique to the chunking being used as a part of the experimental design. The baseline in memory pipeline has a chunk size of 120 words and an overlap of 20 words. The dense milvus, rerank, hybrid and hybrid rerank pipelines have a chunk size of 80 words and an overlap of 20 words. The vector database tuned pipeline has a much larger chunk size at 200 words, but also includes a larger overlap of 40 words. The remainder of the vector database configuration was kept constant in the tuned version such that the effect of chunk size could be evaluated more specifically.

4.4 System Architecture

The developed system follows the common RAG architecture illustrated in Figure 4.1. A user query is used to retrieve relevant document fragments.

These fragments are then inserted into a prompt and provided to the LLM as context. The model then, generates an answer based on the retrieved context and the original question.

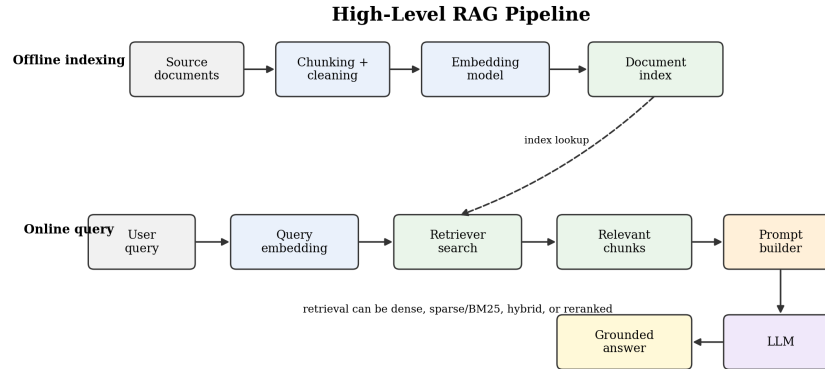


Figure 4.1: High-level RAG workflow used by the implemented notebooks: user query, retrieval over indexed documents, context construction, and answer generation.

Haystack components are used in this application as document conversion, document splitting, document embedding, document retrieval, generating prompts and orchestrating generation workflows. When using a local workflow, Ollama will serve both the answer generation model (llama3.1:8b) and the embedding model (mxbai-embed-large). The milvus based version will access a local milvus instance that is being run by a shell script named `scripts/embed.sh`. A mini-lm cross encoder was used for local reranking of search results. This encoder was trained on MS MARCO ranking data.

The main architectural components are:

- **Document loader and chunker:** Loads .docx files, stores source metadata, and splits documents into retrieval units.
- **Embedding model:** Converts each document chunk and user question into a dense vector representation.
- **Document store:** Stores chunks either in-memory or in Milvus, depending on the pipeline variant.
- **Retriever:** Uses dense vector similarity, sparse lexical retrieval, or hybrid sparse-dense retrieval to identify candidate chunks.

- **Reranker:** Reorders retrieved candidate chunks in the reranking variants before the final context is passed to the prompt builder.
- **Prompt builder and generator:** Combines the retrieved context with the user question and sends the question-context pair to the language model.
- **Results logger:** Writes the implementation name, scenario identifier, retrieved sources, generated answer, and timing values to a shared CSV file.

This structure supports fair comparison because all notebooks use the same high-level workflow while differing at the retrieval or ranking strategy level. It also supports HPC execution because the notebooks and helper scripts can be executed in a managed environment with environment-specific setup changes.

4.5 Implemented RAG Pipelines

4.5.1 Baseline RAG version

The baseline in-memory RAG version, `rag_basic.ipynb`, utilizes an in-memory document store and an embedding-based retriever. The goal of this pipeline was to provide the simplest completely working RAG model and serve as a reference point for the other versions. It demonstrated how retrieval and generation work when there is no added overhead of a vector database, sparse retrieval, or reranking.

4.5.2 Version Using a Vector Database RAG

The vector-database version, `rag_vectordb.ipynb`, places document embeddings into Milvus. For each query, the version queries the Milvus collection and returns the top 10 dense matches. This version tested the effects of utilizing persistent vector databases as opposed to in-memory document stores.

4.5.3 Tuned Version Using a Vector Database RAG

The tuned vector-database version, `rag_vectordb_tuned.ipynb`, has the same Milvus backed dense retrieval structure as the standard vector-database pipeline. However, the primary differences were related to the chunking settings; specifically, the chunk size went up from 80 to 200 words while the chunk overlap increased from 20 to 40 words. This version evaluated whether larger chunks would provide more complete context to the generator and improve answer quality, while also testing the risk that larger chunks may reduce retrieval precision by mixing more content into each retrieved unit. Smaller chunks were not selected for this tuned version because the baseline Milvus configuration already used relatively short 80-word chunks.

4.5.4 Reranking RAG

The "reranking" version (`rag_reranker.ipynb`) performs an initial search for 20 dense candidates with respect to the input using Milvus. It uses a cross encoder reranking model to reorder these candidates and selects the top 10 reranked candidates to be passed to the generator. As opposed to the vector database approach described in Section 4.5.2, this selection is made based on a larger candidate set (of size 20) that was previously generated as the result of an initial search. Therefore, it may help determine if a new relevance ordering step will sufficiently improve the quality of the contextual information provided by the system's previous output to warrant the added time delay for searching the database.

4.5.5 Hybrid Retrieval RAG

The hybrid version, `rag_hybrid.ipynb`, used a hybrid retrieval mechanism made available by Milvus. It combines dense semantic embeddings, which support semantic matching, with a sparse BM25-style representation produced inside Milvus, which supports lexical matching. The goal of this version was to determine if semantic matching and lexical matching could be complementary. This is particularly interesting given that enterprise documents often contain exact term references (e.g., team name(s), severity level(s), product identifier(s), etc.).

4.5.6 Hybrid Retrieval With Reranking RAG

The hybrid-reranking version, `rag_hybrid_&_rerank.ipynb`, built upon the hybrid retrieval version. First, it retrieves 20 hybrid candidates. Next, it uses the cross-encoder reranker to select the top 10 documents for generation. This version may improve context ordering by combining dense retrieval, sparse lexical matching, and reranking, but it also increases retrieval cost: the time needed to perform the retrieval will increase since both hybrid candidate retrieval and reranking occur prior to the actual generation of answers.

Table 4.1 summarizes the six implemented RAG pipeline variants and the components included in each configuration. The comparison highlights how the pipelines differ in terms of document storage, retrieval strategy, chunking configuration, reranking, generation, and logging.

X indicates that the component or configuration is part of the pipeline variant.

Pipeline	In-memory docs	Vector DB	Dense re-trieval	Sparse / BM25	Reranker	Tuned chunks	LLM generation	CSV logging
basic	X	-	X	-	-	-	X	X
vectordb	-	X	X	-	-	-	X	X
vectordb_tuned	-	X	X	-	-	X	X	X
reranker	-	X	X	-	X	-	X	X
hybrid	-	X	X	X	-	-	X	X
hybrid_rerank	-	X	X	X	X	-	X	X

Table 4.1: Implemented RAG pipeline variants and included components.

4.5.7 Measured Variables

This thesis measures the following primary variables.

- Time to index in seconds
- Time to retrieve in seconds
- Time to generate in seconds
- Source files and departments that were retrieved
- Contexts that were retrieved
- Number of retrieved documents

- Top-k retriever and reranker values
- Generated answer text
- Correctness score and label given by manual assessment
- Deterministic measures of answer quality and retrieval quality.

Most of the above-listed variables are logged to a common results log. These include the name of the used implementation, the id of the scenario, the question asked, the answer generated for this question, sources that were retrieved during the search process, departments that were retrieved during the search process, contexts that were retrieved during the search process, the number of documents that were retrieved from the database, the k-values for both retriever and reranker, time spent on indexing, time spent on retrieving information, and finally time spent generating answers. Additionally, manual correctness scores and labels for the answers can be added in an evaluation file provided with the ground truth.

The script for evaluating the answers deterministically will produce some additional quality metrics for the answers and for the retrieval. Time spent per query answering questions can be found in both the retrieval time and generation time. However, as indexing time does represent corpus preparation rather than question-answering latency directly, indexing time is reported separately. Finally, when comparing local vs. HPC systems, the last line of the timing summary includes total runtime for each system's respective pipeline.

4.6 Evaluation Framework

4.6.1 System Performance Metrics

Both local and HPC run times for each experiment will be stored in the results csv file. In both cases, we use identical application-level timing fields (indexing time, retrieval time, generation time, and total execution time).

There are no new quality metrics introduced by HPC evaluation. Instead, we compare the execution time of the same variant pipelines under a different execution environment. Although queue time, service startup time, and data transfer overhead may be important aspects of deploying your system, these were not considered to be separate experimental variables for this study.

4.6.2 Retrieval Quality Metrics

Retrieval quality is evaluated through comparison of the automatically retrieved source documents to the manual selection of supporting documents within the ground truth CSV. The proportion of manually selected supporting documents (out of all) that are retrieved is referred to as "recall". The ratio of manually selected supporting documents (out of all) to the total number of documents retrieved represents "precision". The retrieval F1 score is an average of both precision and recall. For each evaluation, we will be recording a "retrieval hit" if there exists at least one manually selected supporting document included in the automatically retrieved set.

4.6.3 Answer Quality Metrics

Manual evaluations assess the quality of the answer by providing a score to determine if the response was completely accurate, partially accurate, or completely inaccurate. Manual scoring assigns 1.0 for a fully correct answer, 0.5 for a partially correct answer, and 0.0 for an incorrect answer.

The deterministic evaluation script computes additional automatically computed metrics. ROUGE-L is a metric for measuring lexical overlap in addition to semantic similarity between the generated response and the reference response. BERTScore is another metric that utilizes contextual embeddings to compute semantic similarities in addition to lexical overlap between the two responses. Similarly, a sentence transformer can measure the level of semantic similarity between individual sentences within each answer. Additionally, there are metrics related to the retrieval quality of the model such as whether enough supporting documents were retrieved. Lastly, the script will compute the average number of words in each generated answer.

4.6.4 Visual Analysis

Visualizations are produced via a visualization notebook that is specifically designed for visualizing results from which the final answers have been determined and processed. Timing breakdowns (per scenario), answer quality, retrieval quality and performance data per scenario will be shown. These figures are discussed in the results chapter 6 as part of an analysis on trade-offs among answer quality, retrieval quality, and execution time.

4.6.5 Cross-Environment Comparison

The local experiments are run with the notebook workflow with Ollama-based model serving and a local Milvus instance where needed. The HPC experiments are run on the Cyclone HPC [38] facility using Slurm-managed jobs and vLLM-based model serving. Per the available info, Cyclone has CPU-only nodes and GPU-accelerated nodes, which have: 2× 20-core Intel Xeon Gold 6248 CPUs, 4× NVIDIA V100 accelerators, and 192 GB of memory per node.

The implemented HPC workflow launches the model-serving components as separate Slurm jobs.

4.7 Reproducibility and Documentation

Standardization of result collection is achieved through a logging system for all results. However, as already mentioned, the loggers will collect data for each notebook independent of other notebooks. Therefore, there are multiple RAG pipelines being run with their own respective variants. All modules that are necessary for building ground truth and running the deterministic evaluation are separate from the notebooks. This allows to rerun the evaluation module at any time after the experimental data has been generated.

Documentation is necessary for reproducibility. Therefore, there is a README document for setting up the software and running each notebook, along with documenting how to start Milvus and what Ollama models are required, how to execute each notebook, what command to run to perform evaluation, where output files are written etc. The model serving scripts provide an additional layer of documentation of the workflow. In addition, Slurm submission commands and the location of the output CSV files are also documented. Lastly, code comments were added throughout the notebooks and helper scripts to explain why certain implementation steps were taken and what they accomplish.

4.8 Summary

The purpose of this chapter was to provide a detailed description of the methodologies that were employed to build out and evaluate the six RAG

pipeline variants as well as describe the common architectures and systems (dataset, pipeline variants, etc.) used to support testing and evaluation of these pipeline variants.

This chapter also provided an overview of the framework utilized to evaluate each of those pipeline variants with respect to timing, retrieval quality, answer quality, visual analysis, and the differences between local and High-Performance Computing (HPC) environments.

Chapter 5 – Experiments

5.1 Introduction

This chapter details how the experiments were conducted. Chapter 4 described the methodology and evaluation framework; however, it was the focus of this chapter to detail the actual experiment process: software setup, model/service startup, notebook order of execution, question scenarios, parameter settings, logging of results, and output validation.

These experiments have been divided into two phases (Local and HPC) that both run the same six RAG pipeline configurations and the same five scenarios. Local is used to determine baseline performance against which all six RAG pipeline variations will be compared. HPC runs these same six pipeline configurations and the same five scenarios within the Cyclone cluster environment. Although local and HPC workflows do not represent a pure hardware comparison due to differences in the serving stack and backend models between environments.

5.2 Experimental Setup

5.2.1 Software Environment

The experiments were run in a set of Python Jupyter notebooks. Each experiment was executed within its own virtual environment containing all required packages. All the notebooks were run from their respective project notebook directory and contained an identical flow through the various sections described below: Path configuration; Service configuration; Corpus load; Split documents into training sets and test sets; Indexing; Build pipelines; Run scenarios; Log results; Validate results

The executed implementation notebooks are:

- `rag_basic.ipynb`;
- `rag_vectordb.ipynb`;

- `rag_vectordb_tuned.ipynb`;
- `rag_reranker.ipynb`;
- `rag_hybrid.ipynb`; and
- `rag_hybrid_&_rerank.ipynb`.

All notebooks followed a similar general process of execution. That process included importing libraries, configuring paths and services, loading the document corpus, splitting/indexing documents, building the RAG pipeline, executing scenario questions, and saving results. This standardized structure reduces unwanted variation between implementations and makes the pipeline-level comparison more controlled.

5.2.2 Models and Services

The local process utilizes Ollama for model serving. The local generative model utilized is `llama3.1:8b` and the local embeddings model utilized is `mxbai-embed-large`. Both of these models are open-source and may be downloaded via Ollama or model repository sources (e.g., Hugging Face) subject to download restrictions on each model. Each notebook that runs reranked vectors locally utilizing MiniLM cross encoder (sentence-transformers).

Each of the vector-database, reranking, hybrid, tuned-vector-database, and hybrid-reranking notebooks run prior to launch of Milvus, utilize the standalone script `embed.sh` (project script) to initiate the Milvus service. The in-memory baseline is simple to implement, requires no Milvus services to begin with; hence it serves as the simplest baseline implementation.

In contrast, the High-Performance Computing (HPC) workflow utilizes vLLM-based model serving. Hence, the generation service utilized was `meta-llama/Llama-3.1-8B-Instruct`, the embedding service utilized was `BAAI/bge-m3`, and the reranking service utilized was `BAAI/bge-reranker-base`. Therefore, both workflows are based upon the same pipeline structure and timing category, however they are different model-serving deployments. The HPC comparison should be viewed as a comparison at the workflow level and not strictly as an isolated hardware performance metric.

5.2.3 Input Corpus

All experiments make use of the same OrchestrAI synthetic enterprise corpus. The corpus includes 54 DOCX documents divided by departments. Each notebook loads each document recursively and converts it into a Haystack document format; it also keeps track of source information about each document (file name and department).

Since all notebooks have access to the same set of documents, the primary cause for differences in answers is the way in which they retrieve and rank those documents. The same corpus is duplicated on the HPC system used for the second phase and retains the directory structure and file names.

5.2.4 Configuration and Parameters

The key configurable options include the model types, query types (e.g. "what," "where"), total number of retrieved documents, chunking configurations and logging schema. In most cases a total of ten retrieved document-chunks are passed to the language model for evaluation. For the reranked versions, the retriever first returns twenty candidates, and the cross-encoder reranker filters them down to the top ten.

All of the chunking configurations differ from each other only when they were used as part of an experimental design. The baseline used 120 word chunks that had 20 words of overlap. All of the Milvus-supported variants, the hybrid version and the hybrid-reranking variant all used 80 word chunks with 20 words of overlap. The only chunk size used by the vector database tuning notebook was 200 words with 40 words of overlap. The rest of the vector-database configuration remained unchanged.

- **Top 10 retrieval:** basic, vector database, tuned vector database, and hybrid.
- **Top 20 retrieval followed by top 10 reranking:** reranking and hybrid with reranking.
- **Chunking test:** tuned vector database changes only chunk length and overlap.

5.3 Deployment Stages

5.3.1 Local Execution Stage

The local execution stage provides the baseline experimental stage of the project. The primary goal of this stage is to validate that each notebook successfully runs on the author’s computer and to provide comparative timing, retrieval and answer quality outputs.

The notebooks were run in the same order as follows:

1. baseline in-memory RAG;
2. Milvus vector-database RAG;
3. tuned Milvus vector-database RAG;
4. dense retrieval with reranking;
5. hybrid retrieval; and
6. hybrid retrieval with reranking.

This order represents increasing pipeline complexity, starting with the in-memory baseline and then adding persistent vector storage, chunking variation, reranking, hybrid retrieval, and hybrid retrieval with reranking. All outputs generated during each run are recorded into a single `results.csv` file for comparison purposes.

5.3.2 HPC Execution Stage

The HPC execution stage involves the same pipeline sequence and scenario questions as for the local execution stage, with the difference being that model serving is done via vLLM, with generation, embedding, reranking services, and that execution is through Slurm-managed jobs.

The HPC experiments were run on the Cyclone HPC facility. The service launch scripts submit separate Slurm jobs for the generation model, the embedding model, and the reranking model. Each service job requests 1–node, 1–GPU, 1–task-per-node, and 10–CPU cores.

The vLLM generation service uses `meta-llama/Llama-3.1-8B-Instruct`; the embedding service uses `BAAI/bge-m3`; the reranking service uses

BAAI/bge-reranker-base. Each service then writes its host, port, URL, API key, and model name into an environment file under `/nvme/scratch/${USER}`. Finally, these environment files are used by the HPC notebooks to connect to the right model-serving endpoints.

The purpose of this stage is to compare application-level timing behaviour across environments. Queue time, service startup time, and data-transfer time are deployment issues worth considering, but they won't be treated as separate measured variables in this thesis. The focus is on timing comparisons of indexing time, retrieval time, generation time, and total runtime.

5.4 Experiment Scenarios

5.4.1 Scenario-Based Query Execution

Five predetermined question-answering scenarios serve as the framework for the experiments. Different retrieval and reasoning behaviours are tested in each case. A straight factual lookup is the first scenario. The latter situations call for cross-document or procedural reasoning.

The selection of five scenarios provides a focused but diverse comparison set rather than a statistically exhaustive benchmark. The scenarios were chosen to represent common enterprise question types: single-fact lookup, procedural checklist retrieval, cross-team operational reasoning, policy exception handling, and multi-hop customer-risk analysis. Using the same five scenarios across all six implementations keeps the evaluation controlled while still testing whether each pipeline handles different retrieval and reasoning demands.

The five scenario questions are:

1. ***What is the first-response SLA for a Sev-1 support incident?***

This is the clean baseline scenario. It checks whether the system can retrieve and answer a single operational fact accurately.

- Retrieval type: single-document
- Main test: exact fact retrieval

2. ***What steps must be completed before a new employee is considered ready for day one at OrchestrAI?***

This scenario is the broadest scenario because it requires the system to

connect support, product, logistics, and commercial-risk information into one operational response.

- Retrieval type: checklist or procedure retrieval
- Main test: whether the system can recover and summarize a structured process

3. ***A new engineer starts on Monday, but their laptop has not arrived and their required access is still pending. Which teams are involved and what should happen next?***

This scenario requires combining information from more than one team to answer a realistic workplace problem.

- Retrieval type: multi-document, cross-team retrieval
- Main test: whether the system can synthesize HR, IT, and access-related responsibilities

4. ***Can OrchestrAI make an urgent hardware purchase without following the normal PO workflow if a warehouse outage is at risk?***

This is the most demanding scenario in the notebook because it requires cross-functional reasoning across several business areas.

- Retrieval type: policy retrieval with exception logic
- Main test: whether the system can identify when a standard workflow may be bypassed

5. ***A customer renewal is at risk because support cases are recurring, a requested feature is still unavailable, and replacement hardware is delayed. What should OrchestrAI do next, and which teams must be involved?***

This scenario is slightly more complex because the answer is not a single fact but a short operational process.

- Retrieval type: multi-hop, multi-document retrieval
- Main test: whether the system can combine support, product, logistics, and commercial risk signals into one grounded answer

The same questions are used for all implementations. Each implementation uses its own scenario identifier prefix so that rows remain easy to distinguish in the results file. For example, the baseline uses b1–b5, the vector-database run uses v1–v5, the hybrid-reranking run uses hr1–hr5 etc.

5.4.2 Baseline RAG Experiment

In this baseline experiment, the five scenarios are executed with the in-memory RAG pipeline. The DOCX chunks are loaded in memory and indexed into an in-memory document store. For each of the five questions, the system retrieves the top 10 dense match candidates. This experiment serves as a reference point for all other experiments presented in this thesis.

5.4.3 Vector Database RAG Experiment

This experiment executes the same five scenarios, but now on a persistent vector database (Milvus). For each of the five questions, the system searches for the top 10 dense match candidates. This experiment examines the impact of persistently storing vectors in a database and whether the quality of the retrieved matches stays similar to those produced by the baseline experiment.

5.4.4 Tuned Vector–Database RAG Experiment

The following experiments execute the same five scenarios by running the Milvus vector-database pipeline at a larger chunk size with higher overlaps. Chunk size will impact how much context is included for each retrievable unit. To remove any other influences from the chunk-size on the retrievals (i.e., to simply determine the effect of the chunk-size), all else constant are kept; that is, the retriever, model, prompt and top-k values are identical to those used within the vector-database configuration. Thus, these experiments are referred as the "tuned" variant of the vector-database experiments.

5.4.5 Reranking Experiment

For this reranking experiment, the five scenarios are executed using dense retrieval (Milvus) followed by cross-encoder reranking. First, the retriever delivers 20 candidates and then the reranker selects the best 10 candidates for prompting. This experiment assesses the trade-off between additional latency during the retrieval stage versus improved ranking order.

5.4.6 Hybrid Retrieval Experiment

The hybrid retrieval experiment combines dense semantic retrieval with a sparse BM25-style signal generated inside Milvus. It retrieves 10 documents for each question. This experiment evaluates whether combining semantic and lexical signals improves robustness for questions that contain exact operational terms such as severity labels, team names, procurement terms, and workflow names.

5.4.7 Hybrid Retrieval with Reranking Experiment

In the hybrid reranking experiment, cross-encoder reranking is applied after hybrid candidate retrieval. The hybrid retriever provides 20 candidates and the reranker chooses the best 10. This is the most complex pipeline currently implemented. This experiment tests whether hybrid recall combined with reranked precision generates better answers than previous implementations at a higher computational cost.

5.5 Benchmarking and Data Collection

All outputs will be logged into the project's results logger. Before each run of the notebooks, rows for the respective implementations will be deleted so that multiple runs will replace previous data instead of producing duplicate data points. Every row will contain the name of the implementation, ID of the scenario, question, answer provided by the system, source documents, departmental information based on available documentation, available context, Number of documents available, top-k parameters, time it took to index the documents (one-time), time it takes to retrieve answers (query-time) and time it takes to generate an answer (query-time).

In the last local vs HPC comparison of timing, these columns will be utilized to create a total time run and speed up analysis for each pipeline. Once all the scenarios have been ran with their respective pipelines, the results file will be used to create the manual evaluation worksheet. Using evidence from the DOCX corpus, scores and supporting documents will be populated into the manual evaluation worksheet. The deterministic evaluation script will calculate the answer quality metrics and retrieval quality metrics and create per-scenario as well as summary CSV files. The visualization notebook will

utilize these processed outputs to generate plots of answer quality, retrieval quality, scenario-based analysis, timing-based analysis and local vs. HPC comparison for chapter 6

5.6 Data Validation

The recorded output is validated as complete prior to analysis. The validation requires verification that there are five rows in each of the implementations, that the generated answers were not null, that there were sources that were retrieved, and that the time fields were available. Additionally, all retrieved contexts are manually examined when developing the truth table (ground-truth sheet), since the quality score of the answer cannot be assumed; it must have some form of evidence based upon a subset of the data within the corpus.

For the HPC experiments, validation focuses on whether the same six pipeline variants and five scenarios are present in the timing outputs and whether the timing fields required for local versus HPC comparison are available.

5.7 Summary

This chapter outlined how the experimental setup and implementation were carried out for all of the six developed RAG pipelines. It also discussed the environment where the software was executed, the model-serving setup used to deploy models, the input corpus, the parameters that were set during execution, the sequence in which notebooks are run, the phases of deploying an application, the types of question asked by each scenario, how data was collected from each experiment, and what steps were taken to validate the results of experiments conducted with this pipeline.

Since both environments utilize different model-serving back-ends and some different back-end models, when comparisons are made across environments they should be viewed as a comparison of workflow execution times rather than a hardware comparison. Data generated in this way is then analyzed in chapter 6

Chapter 6 – Results and Discussion

6.1 Introduction

This chapter presents the outcomes from the two stages of execution (local and HPC) of this work. The local stage is concerned with measuring the time for the three types of processing (retrieval, quality of answers and overall run-time). These are detailed within chapters 4 and 5. A comparative analysis will be undertaken to determine how well each of these stages has performed relative to one another in terms of their run-times.

At the local level it can be seen that the performance of the three elements of the pipeline (retrieval, quality of answers and run-time) do not necessarily progress alongside. At the HPC level we will compare the run-time performances of all six variations of the pipeline in an alternative execution environment.

6.2 Results of the Local Runs

For the six local versions the averaged results are shown in Table 6.1.

Implementation	Manual score	Ret. F1	Index (s)	Retrieve (s)	Generate (s)	Answer len. (words)
Basic	0.80	0.424	17.65	0.067	27.47	147.0
VectorDB	0.60	0.451	35.43	0.104	21.95	104.0
VectorDB tuned	0.60	0.366	36.87	0.103	31.67	120.8
Reranker	0.50	0.345	57.69	1.148	28.31	85.8
Hybrid	0.40	0.279	34.87	0.175	25.52	116.4
Hybrid rerank	0.60	0.405	62.96	2.988	35.79	119.0

Table 6.1: Summary of local evaluation results averaged across the five scenarios.

As indicated by Table 6.1, the best manual answer score and the best retrieval F1 were obtained by different pipelines. This shows that answer quality and retrieval quality are related but not equivalent. A pipeline may retrieve relevant evidence but still generate an incomplete answer if the model does not select the required operational details. On the contrary, a pipeline may produce a strong answer in a small scenario set even if its retrieval metrics are not the highest overall.

6.2.1 Local Timing Comparison

Figure 6.1 illustrates the average execution time for each local run. The simplest in-memory pipeline indexed the corpus fastest, with an average indexing time of 17.65 seconds. The Milvus-supported pipelines took longer to index because document chunks and embeddings had to be prepared and stored through the vector-database backend. The reranking and hybrid with reranking pipelines had the longest indexing times in the local results, which reflects the additional setup and component initialisation required by the more complex pipeline configurations.

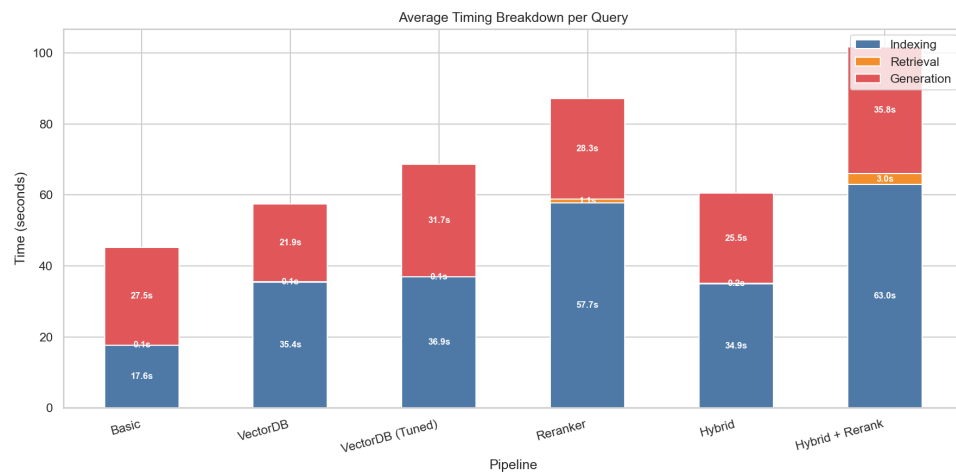


Figure 6.1: Average local timing breakdown for indexing, retrieval, and generation across the implemented RAG variants.

Retrieval times were very short for the basic pipeline, the vector-database pipeline, and the tuned vector-database pipeline. The basic pipeline took 0.067 seconds on average, the vector-database pipeline took 0.104 seconds, and the tuned vector-database pipeline took 0.103 seconds. Hybrid retrieval increased average retrieval time to 0.175 seconds. Reranking caused a larger increase because an additional cross-encoder scoring step was applied to the candidate set. The longest retrieval time was produced by the hybrid with reranking pipeline, with an average of 2.988 seconds.

The generation times for each implementation were greater than the retrieval times for each implementation. The vectorDB pipeline had the lowest average generation time with an average of 21.95 seconds and the hybrid with reranking pipeline had the largest average generation time with an average of 35.79 seconds. This indicates that in this local environment, the language-model generation step dominated the per-query response cost. Thus, although retrieval optimization is important, the latency difference was substantially

less than that due to generation latency, other than when using a hybrid with reranking pipeline.

6.2.2 Local Retrieval and Answer Quality

Figure 6.2 compares the local answer-quality metrics. The basic pipeline received the highest manual score, with an average of 0.80. The vector-database, tuned vector-database, and hybrid with reranking pipelines each achieved a manual score of 0.60. The reranking pipeline scored 0.50, while the hybrid pipeline had the lowest manual score at 0.40.

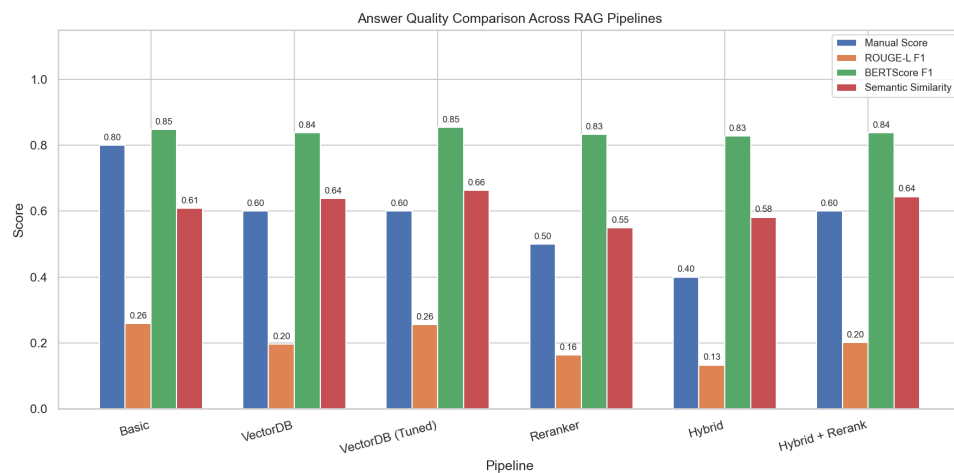


Figure 6.2: Answer-quality metrics for the local RAG variants, including manual score, ROUGE-L, BERTScore, and semantic similarity.

The results from using automatic semantic metrics did not always match up perfectly to the manual scores. For example, although the tuned version of the vector-database pipeline got the highest BERTScore and semantic-similarity (which both reward how much information overlaps with a given reference-answer) it only received a manual score of 0.60. There are two reasons for this discrepancy. First, semantic similarity only looks at how much information in the generated response overlaps with the information in the reference answer. Second, manual scoring checks whether the answer provides enough operational detail or leaves out required steps. Therefore, an answer can score well semantically while still receiving a lower manual score if it omits required details needed to complete the operation correctly.

As shown in Figure 6.3, the vector-database pipeline produced the best results for all retrieval-based metrics. It scored an F1 of 0.451, which represented the highest value for that metric; and it also scored a precision of 0.319, again representing the highest value for that metric. The basic-pipeline

scored an F1 of 0.424, and hybrid-reranking scored an F1 of 0.405. All four of those pipelines (basic-pipeline, vector-database pipeline, tuned vector-database pipeline, and hybrid with reranking) scored a perfect 100% on retrieval-recall. The reranker scored 90%, and the hybrid-pipeline scored 70%.

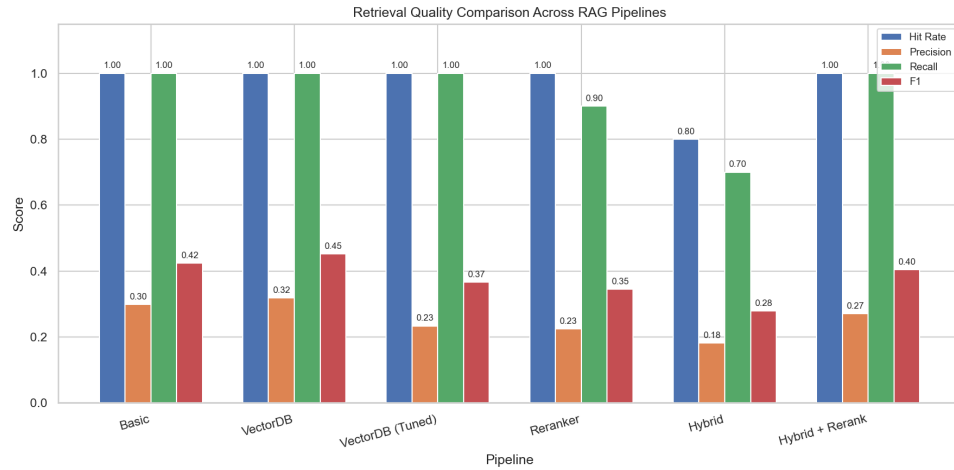


Figure 6.3: Retrieval-quality metrics for the local RAG variants, comparing hit rate, precision, recall, and F1.

The results from this local test indicate that hybrid pipeline performance did not improve overall search quality. Hybrid search is expected to improve performance as a combination of both lexical and semantic cues can enhance search results. However, neither the Milvus hybrid configuration nor the small number of scenarios used in this study provided a significant improvement over the dense vector-database pipeline.

The hybrid with reranking variant improved retrieval quality in comparison to the hybrid-only variant; however, there was also a large increase in retrieval time.

Manual score-by-scenario in Figure 6.4 shows that scenario 1, Sev-1 SLA Lookup, was the least difficult (average score = 0.92). Scenario 3 and 5 were the most difficult (both averaged 0.42) as they required cross-functional reasoning about onboarding, IT, Logistics, Customer Success, Product and Commercial processes. Low scores suggest that searching for the correct documents is insufficient if the user needs information regarding multiple operationally dependent processes.

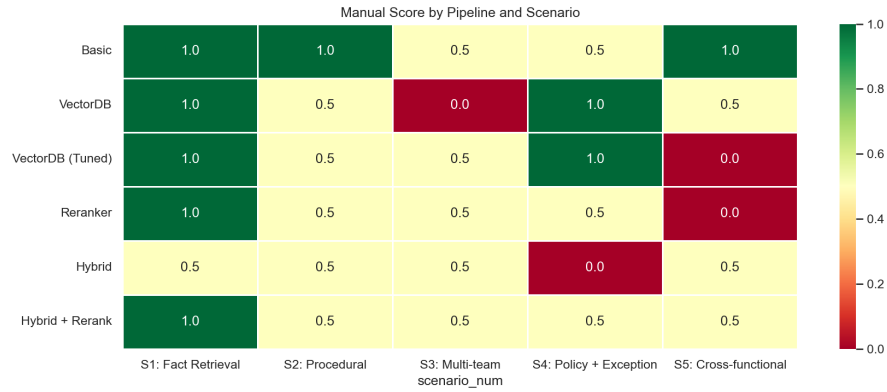


Figure 6.4: Manual answer score by implementation and scenario. Darker cells indicate stronger manually evaluated answers.

To provide a broader comparison of pipeline behavior across multiple evaluation dimensions, Figure 6.5 visualizes the performance profile of the local RAG pipeline variants.

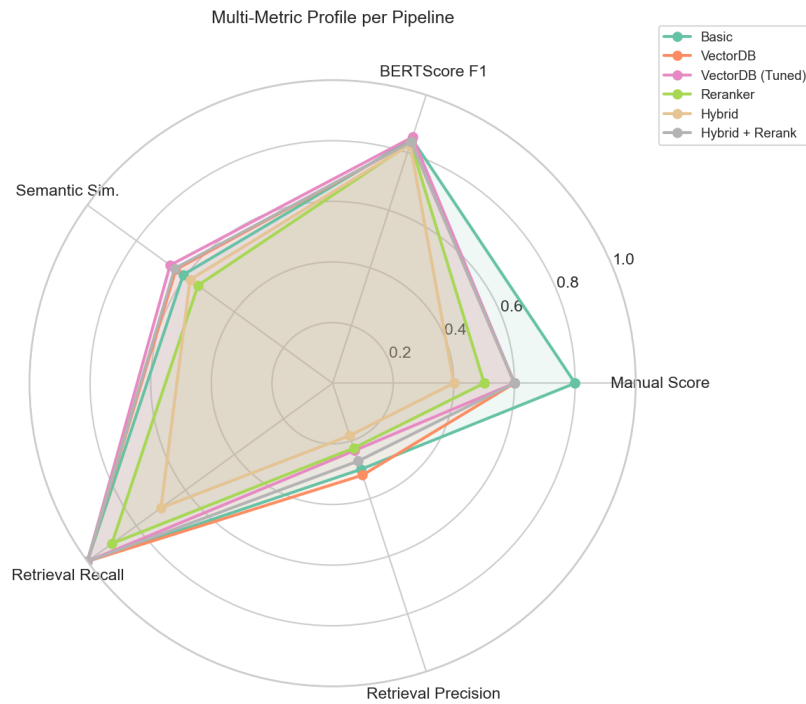


Figure 6.5: Multi-metric profile of the local RAG pipelines, comparing manual score, BERTScore, semantic similarity, retrieval recall, and retrieval precision.

6.3 HPC Deployment Results

The HPC phase ran the same six RAG pipeline architectures (and the five scenario question types) that were used in the local workflow. This section's comparison will be limited to the timing data collected by the two

workloads. Because the local and HPC workflows are running with differing model-serving backends, and differing backend models for some of them, the comparison will be viewed strictly as a workflow level timing study, and not as a hardware-based benchmark.

For all six pipelines, generation time was significantly better when the HPC workflow was deployed. Generation speed-ups compared to the local workflow ranged from 7.390 $\times$ for the hybrid pipeline to 10.518 $\times$ for the tuned vector-database pipeline. Indexing time was also better for each of the pipelines run; however, the best improvement in indexing speed was seen in the basic pipeline at a 10.164 $\times$ speed-up.

Retrieval time did not behave as predictably. The basic, reranking, and hybrid-reranking pipelines performed faster on HPC in terms of retrieval time. Conversely, the vector-database, tuned vector-database, and hybrid pipelines performed worse (slower) on HPC.

6.4 Local versus HPC comparison

Table 6.2 summarizes the local-versus-HPC timing comparison. Speed-up is calculated as local time divided by HPC time, so values greater than 1 indicate faster execution on HPC and values below 1 indicate slower execution on HPC.

Pipeline	Local total (s)	HPC total (s)	Index speed-up	Retrieval speed-up	Generation speed-up	Total speed-up
Basic	45.188	4.955	10.164$\times$	2.454 $\times$	8.609 $\times$	9.120$\times$
VectorDB	57.479	16.362	2.559 $\times$	0.369 $\times$	9.810 $\times$	3.513 $\times$
VectorDB Tuned	68.640	16.017	2.871 $\times$	0.623 $\times$	10.518$\times$	4.285 $\times$
Reranker	87.150	18.004	4.193 $\times$	2.412 $\times$	7.512 $\times$	4.841 $\times$
Hybrid	60.563	18.453	2.420 $\times$	0.296 $\times$	7.390 $\times$	3.282 $\times$
Hybrid + Rerank	101.738	19.343	4.257 $\times$	3.819$\times$	9.489 $\times$	5.260 $\times$

Table 6.2: Local versus HPC timing speed-up summary.

In terms of total runtime, HPC execution reduced runtime for every pipeline. However, the component-level speed-ups varied. Indexing and generation improved across all pipelines, while retrieval showed the most variability. Retrieval speed-up values below 1 $\times$ indicate that retrieval was slower on HPC. This occurred for the vector-database, tuned vector-database, and hybrid pipelines. These results suggest that vector-database access, service interaction, storage behaviour, or network overhead can outweigh the benefit of HPC resources for short retrieval operations.

Figure 6.6 visualizes the total runtime comparison between local and HPC execution for each pipeline.

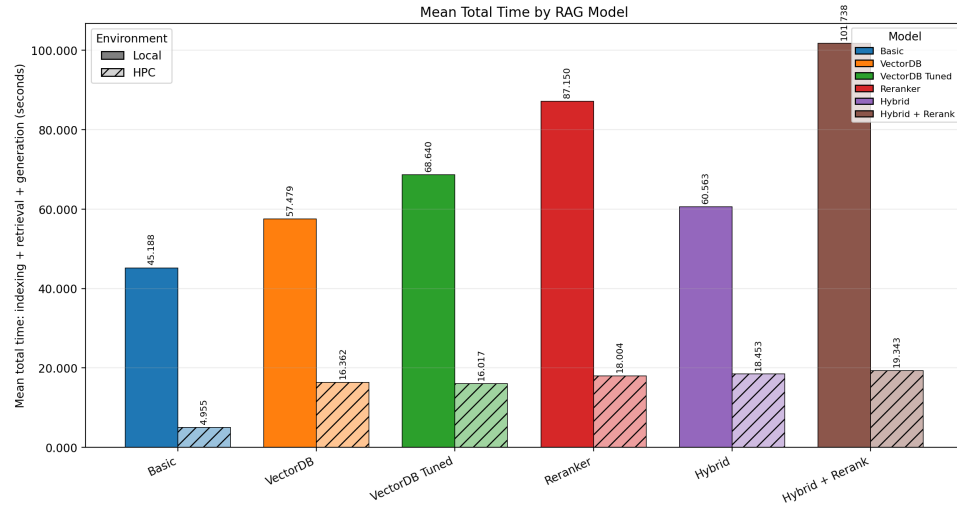


Figure 6.6: Mean total runtime for each RAG pipeline in the local and HPC environments.

Figure 6.7 shows the component-level speed-up patterns. Values greater than $1\times$ indicate faster execution on HPC, while values below $1\times$ indicate slower execution on HPC.

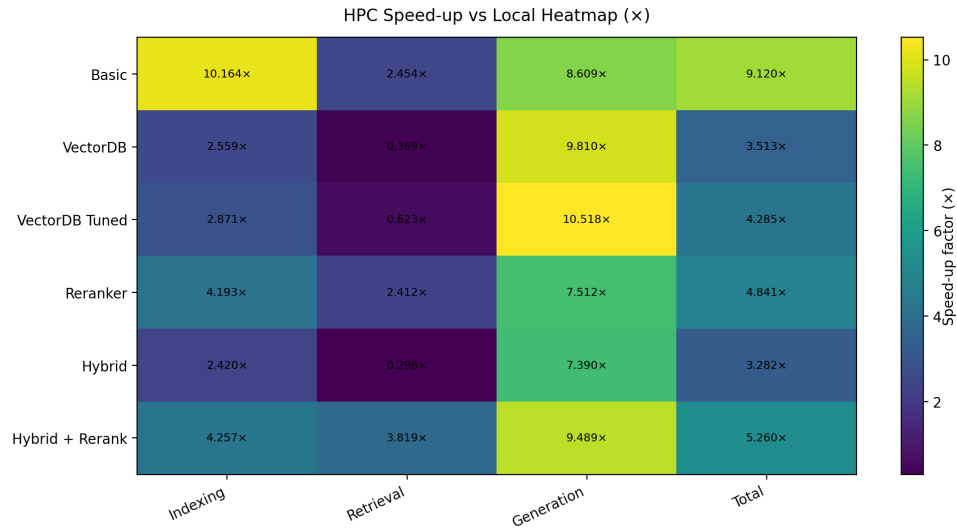


Figure 6.7: Speed-up heatmap comparing local and HPC execution across indexing, retrieval, generation, and total runtime.

6.5 Effectiveness of Retrieval-Level Optimization Techniques

Local results showed mixed effectiveness for retrieval-level optimization techniques. Dense retrieval using Milvus-backed vectors enhanced local F1 scores compared to the in-memory baseline but did not improve manual

answer score. This suggests that storing vectors persistently provides a stronger foundation for retrieving data without always enhancing generative quality.

The tuned vector-database run yielded better ROUGE-L, BERTScore and semantic similarity than the original vector-database run; however it also decreased local retrieval F1 score. Larger chunks may have given the generator more complete context while also making source-level retrieval less accurate. This supports treating chunk size as an experimental parameter with meaning, rather than just a technical process choice.

Reranking did not enhance local manual score in this particular experiment and caused average retrieval latency to increase from the sub-second range to more than one second on average. The hybrid-reranking pipeline was better than hybrid retrieval alone and equivalent to the manual score of the vector-database variant, but had the longest times for both retrieval and generation. Thus, reranking introduced a clear cost, while its quality benefit was limited for this corpus and scenario set.

The results of the HPC timing also make a significant point about the overall (local) level of retrieval performance. The basic pipeline achieved the greatest total speed-up during HPC execution; however, this is not to say that the basic pipeline will generate better retrieval quality than the other pipelines; rather, it merely generated the largest benefit in terms of runtime as a result of HPC execution. Likewise, both the generation and indexing times were improved by HPC on the Milvus-based pipelines, which contributed to their respective improvements in total runtime, whereas the retrieval time in each of the VectorDB, VectorDB-Tuned, and Hybrid pipelines was actually longer than before. In conclusion, these results support the notion that at the retrieval level, optimizations and costs associated with execution are distinct.

Figure 6.8 shows how indexing, retrieval, and generation contribute to total runtime in both environments.

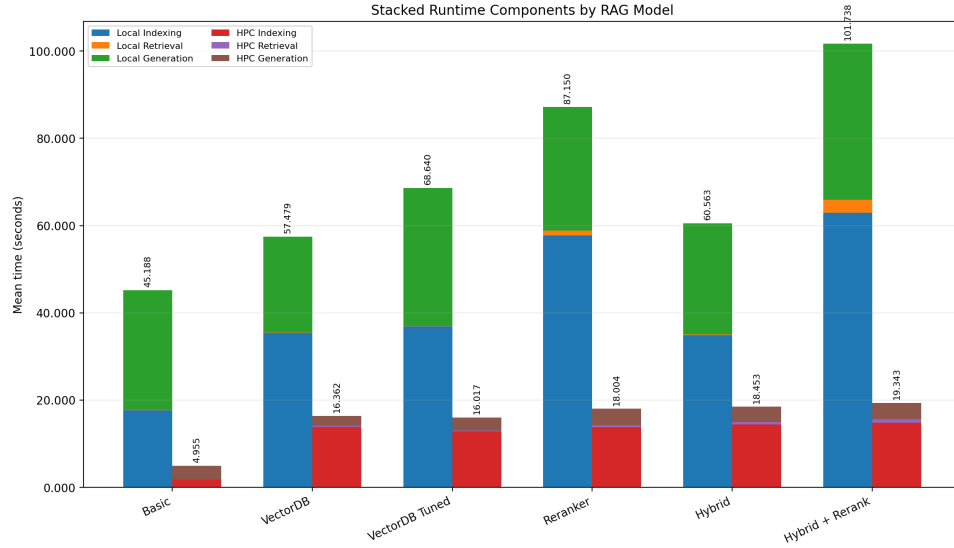


Figure 6.8: Runtime component breakdown for local and HPC execution.

6.6 Research Questions and Hypotheses

The results address the research question through isolating the local retrieval/answer-quality findings from the local vs. HPC timing findings.

For **RQ1**, the results show that retrieval techniques have an impact on both retrieval quality and answer quality; however, they are not always positive. The vector-database pipeline produced the highest retrieval F1 value, while the basic pipeline produced the highest manual answer score. Thus, **H1 is partially supported**: Changing the retrieval pipeline will positively affect both retrieval and answer outcomes; however, hybrid retrieval and reranking did not produce the best overall local results.

As for **RQ2**, the results confirm a previously established performance trade off. Reranking and hybrid with reranking added to the retrieval latency for each of the basic and vector-database pipelines. Hybrid with reranking outperformed hybrid retrieval as far as retrieval F1 goes, but it also resulted in the longest retrieval time. Thus, **H2 is supported**: Adding additional retrieval stages increases the retrieval costs. Further, simply increasing the strength of a retrieval component does not automatically lead to faster answers or higher quality scores.

As far as, for **RQ3** the timing data show that in nearly all of the local pipeline configurations, generation time accounted for most of the overall query time. The retrieval time was a fraction of the generation time for the basic, vector-database, tuned vector-database and hybrid pipelines; however, as reranking

was enabled, the retrieval time became more important. Additionally, indexing time differed significantly among pipelines due to more complex configurations requiring further initialization, additional vector storage, etc. Thus, **H3 is supported**.

Finally, for **RQ4**, the local versus HPC results indicate that all six pipelines were able to achieve a lower total runtime using HPC. As far as total speed-up goes, an average total speed-up of 5.05x can be observed with the largest individual speed-up being associated with the basic pipeline. However, the per-component results demonstrate that HPC was not equally beneficial for all of the timing categories. Specifically, for three out of four pipelines (vector-database, tuned vector-database, hybrid pipelines) slower times were noticed for retrieval within HPC. Thus, **H4 is partially supported**: HPC execution positively impacted total runtime and generation heavy workload; however, it did not positively impact retrieval time for all of the pipeline variations.

6.7 Comparison with Literature

The results confirm literature surveyed in chapter 3; which stated that RAG performance is dependent upon both retrieval quality (the accuracy of what has been found) and generation behaviour (how good the generated answers are). On the local results, dense vector retrieval had good results due to the fact that the most likely source of evidence for policy, handbooks, and operation documentation contained semantically related content. But high retrieval scores didn't always correlate with the highest manual answer scores. Therefore, the idea of evaluating RAG based on both the quality of the retrieved evidence and the quality of the generated answer has merit.

The reranked results are consistent with the typical tradeoff of increased latency for cross-encoder reranking. Reranking is able to enhance candidate ordering by utilizing a more relevant model once candidates have been selected during the initial retrieval phase. However, reranking introduces another processing phase into the overall pipeline. During this thesis, this additional processing introduced significant increases in the time required to complete the retrieval phase of the pipeline, however, it resulted in limited improvements to manual answer quality when considering a very small corpus size and a very small number of test scenarios.

The results from the hybrid retrieval experiments were somewhat unex-

pected. Hybrid retrieval is typically presented as a method to provide exact lexical matching combined with semantic retrieval. However, the local results demonstrated that hybrid retrieval will require extensive tuning before it can be reasonably assumed to produce better results than dense retrieval. A variety of factors including query type, sparse vector configurations, document structures and fusion parameterization can all impact the ability of hybrid retrieval to improve results.

The HPC timing results support discussions surrounding deployment and model-serving overheads provided in chapter 3. Large generation time improvements demonstrate that the HPC serving environment is advantageous for generating heavy workloads. However, slower times for many of the Milvus-based variants demonstrates that HPC may not be equally advantageous for all pipeline components. Shorter retrieval operations may result in service interaction times, storage access times or other vector-database overhead times dominating measured times.

6.8 Trade-offs and Limitations

There is one major trade-off involved in this study; namely, a balance between ease of implementation and representation of real-world deployment. The in-memory pipeline (basic) was implemented most quickly and was able to produce the best manual score with the shortest indexing time; however, it is less indicative of a scalable production-based retrieval system. On the other hand, the vector-database pipeline had the longest indexing time, but provided the largest improvement over baseline retrieval (i.e., the strongest retrieval F1) and can serve as an example of a more realistic storage base for larger document collections.

Another important trade-off involves retrieving relevant documents versus increasing the time required for queries. reranking and hybrid with reranking both added steps to evaluate each document’s relevance. Although the latter technique improved retrieval quality compared to hybrid retrieval alone, it resulted in the longest retrieval times. Therefore, whether or not to pay this additional cost will depend upon how valuable additional retrieval accuracy will be in a particular application setting.

The results from this thesis have several limitations that restrict their generalizability. First, the corpus used is synthetic and there are only five questions in the question-set used in this study. As such, generalization of these re-

sults to all possible scenarios is impossible. Second, manual scoring uses a very simple rubric and therefore likely did not fully account for the nuances of answer quality. Finally, the local results are dependent upon the state of the local runtime environment (e.g., Ollama behaviour, model warm up, background processes, etc.) as well as the Milvus service.

The major limitation of the HPC comparison is that it focused solely on runtime. Thus, while it allows for drawing conclusions regarding indexing time, retrieval time, generation time, and total runtime; it does not allow making conclusions regarding answer quality or retrieval quality. Furthermore, the timing summary provided by the local and HPC workflows do not separate out queue time, service startup time, data transfer time and detailed hardware utilization. Additionally, the local and HPC workflows utilize different serving stacks and some different backend models; therefore the comparisons made here should be considered at a workflow level deployment comparison rather than a pure hardware comparison.

6.9 Summary

This chapter has shown the local and HPC results of the six implemented RAG pipeline variants. The local analysis revealed there is no single pipeline which achieves the best results at each level (i.e., answer quality, retrieval quality, run-time).

The HPC timing comparisons demonstrated that all six pipelines exhibited significantly faster total runtime when executed using HPC resources, although the greatest reductions in processing time were observed for the generation time. Conversely, the degree to which each of the pipelines benefited from the use of HPC for reducing the amount of time required for retrieving answers varied widely. Moreover, it appears that both the process of accessing stored vectors within databases, as well as the overhead associated with interacting with services can be a significant impediment to rapid retrieval times for relatively short queries.

In summary, the results provide evidence supporting the primary research objective of this thesis that optimizing the performance of retrieval-level RAGs is contingent upon a variety of trade-offs regarding the quality of answers produced by these systems, the quality of retrievals provided by them, and costs incurred during system execution. Furthermore, the results indicate that executing pipelines via HPC methods can potentially reduce

overall runtime and may produce greater benefits for those pipelines where generation time dominates overall processing time; however, simply executing pipelines via HPC does not guarantee improved performance for each individual component.

Chapter 7 – Conclusion and Future Work

7.1 Introduction

This chapter presents a summary of the major results that were obtained through this project. It will identify the contributions as well as provide information regarding the limitations associated with this thesis. Additionally, it will outline possible areas for further development related to this research area.

In addition to investigating the behaviour of six different RAG-pipeline-variants at the level of retrieval, this research study measured both local performance of these variants. Furthermore, it also compares the local performance of each of the six RAG-pipelines using time-based metrics in a local environment and using time-based metrics in a High-Performance Computing (HPC) environment.

7.2 Summary of Findings

Local experimentation demonstrated that the in-memory pipeline performed best by manual evaluation of answers; however, the vector-database pipeline produced the highest value of retrieval quality metric (F1). This shows that answer quality and retrieval quality are correlated, yet not equivalent. A pipeline may successfully retrieve relevant documents, and therefore produce a relevant document set, yet fail to provide a complete answer due to omission of necessary operational detail in its generative component.

Timing results indicated the same trade-off exists between pipeline complexity and processing cost as was previously determined with regard to retrieval time. Both basic and vector-database pipelines exhibited very low retrieval latency. However, reranking and hybrid-reranking introduced additional relevance-score stages that caused the additional processing costs associated with these stages to increase the overall retrieval time. Hybrid retrieval alone did not outperform dense vector retrieval in the local experiment. There-

fore, it is likely that hybrid with retrieval will require careful parameterization prior to it being able to be considered as providing improved retrieval quality.

In comparing the local vs HPC environments, all six pipelines produced lower total run times when running in the HPC environment than in the local environment. The average total speed-up was approximately 5.05x. The greatest improvement was seen in generation time, whereas retrieval-time improvement was inconsistent. Additionally, some Milvus-based variants demonstrated slower retrieval times in HPC. This indicates that service interaction, vector database access and storage behaviour may have significant effects on shorter duration retrieval operations.

Table 7.1 summarises how the findings answer the research questions and hypotheses.

RQ	Main finding	Hypothesis outcome
RQ1	Retrieval design affected both retrieval quality and answer quality. The best retrieval F1 and best manual answer score were achieved by different pipelines.	Partially supported
RQ2	Reranking and hybrid reranking increased retrieval latency. Hybrid reranking improved retrieval quality compared with hybrid retrieval alone, but had the highest retrieval time.	Supported
RQ3	Generation time dominated query-time cost in most local pipelines. Retrieval time became more important when reranking was added.	Supported
RQ4	HPC execution reduced total runtime for all six pipelines, but did not improve every timing component uniformly.	Partially supported

Table 7.1: Summary of research-question answers and hypothesis outcomes.

7.3 Key Contributions

The main contributions of this thesis are:

- a focused explanation of LLM application approaches, including prompting, RAG, fine-tuning, and training from scratch;

- a practical implementation of six comparable retrieval-level RAG pipelines;
- a controlled local benchmark comparing answer quality, retrieval quality, and timing across the six pipelines;
- a reproducible notebook-based workflow with shared result logging and deterministic evaluation scripts;
- a scenario-based evaluation framework using a synthetic enterprise corpus and manually selected supporting evidence;
- a local-versus-HPC timing comparison of the same six pipeline structures; and
- an analysis of the trade-offs between retrieval design, answer quality, retrieval quality, and runtime cost.

7.4 Limitations

Some important constraints to bear in mind for this thesis include: firstly, the evaluation corpus is artificial and of limited size. It has been constructed to represent typical corporate style documents, although it does not demonstrate the full extent of either the size, noise levels or diversity that can exist within actual commercial document collections.

Secondly, the evaluation examines five distinct application scenarios. In each scenario, the queries are applied to differing types of search tasks (direct lookup, procedural search, multi-disciplinary reasoning, security policy retrieval, and operational support) and while these do provide a good range of application-based searches, there are insufficient numbers of scenarios to allow for extensive statistical generalisation.

Thirdly, manual scoring employs a simplistic "correctness" based metric. While this provides complete transparency into how answers have been scored, it is likely that such a method will fail to accurately assess many other aspects of answer quality that could potentially impact on user experience, such as the tone, completeness, confidence etc., of answers provided at the sentence level.

Lastly, both the local versus HPC comparisons are workflow level deployments, as opposed to being pure hardware benchmarks. In addition, the

workflows employ similar pipeline structures and timing categories. However, the two workflows utilise fundamentally different server-side architectures and some differing back-end model configurations. Hence, whilst the HPC timings are indicative of the timing characteristics exhibited by the implemented workflow architecture, they are not independent measures of pure hardware performance.

7.5 Future Work

Possible extensions to this thesis can also lead into many other directions. The first area for extension is that the proposed evaluation should be conducted on a much larger and more realistic collection of documents than were used here. As an example, the use of noisy or very long documents and/or documents representing various organizational structures (i.e., hierarchical) will allow for determining if the previously retrieval patterns hold true in a more practical application.

The second area for extension would be to increase the number of scenarios included in the current five. An increased number of test scenarios will provide a basis for better statistical analysis as well as provide a means to easily compare answers across various categories.

A third potential area for extension would involve comparing larger models within the same model family in the HPC environment. The current thesis did not include this type of evaluation, however, investigating how much the increased processing power available in the HPC environment increases the quality of answers produced by larger models is important.

A fourth area for extension is to design a strict hardware-only comparison where all aspects of the model-serving stack, the embedded model, the reranking model, and the vector database are identical in both local and HPC environments. Such a comparison will facilitate isolating the effects of the execution environment on RAG behaviour.

A fifth potential area for extension involves recording other deployment metrics such as queue time, service start-up time, data transfer times, memory utilization, GPU utilization and through-put while responding to concurrent queries. Recording these additional metrics will enable a more comprehensive understanding of RAG behaviour in real-world HPC deployments.

Finally, there are a number of areas that need further investigation regarding

automated RAG evaluation techniques. Specifically, we wish to investigate faithfulness scoring, citation accuracy, hallucination detection and human evaluation involving multiple annotators.

7.6 Final Remarks

This thesis that Retrieval-Augmented Generation (RAG) involves trade-offs between retrieval quality, answer quality, and runtime cost. The experimental results show that stronger retrieval components do not automatically produce the best manual answer quality, and that additional retrieval stages can increase latency even when they improve retrieval coverage.

The HPC results show that moving the workflow from a local machine to an HPC environment can dramatically reduce overall time for a run; particularly for workload heavy on generative processes. Nevertheless, the results also show that each component of the process has its own optimal level of benefit. For example, retrieval-based operations including vector database access or service interactions can experience significant overheads which cannot simply be reduced through use of a larger environment to execute in.

In general, this research provides a framework to reproduce comparison of variants of RAG pipelines within local execution environments as well as HPC execution environments and demonstrates the value of considering both quality and performance.

Bibliography

- [1] J. Alammam, “The illustrated transformer,” <https://jalammar.github.io/illustrated-transformer/>, 2018.
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [3] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, 2020. [Online]. Available: <https://arxiv.org/abs/2005.14165>
- [4] J. Kaplan, S. McCandlish, T. Henighan *et al.*, “Scaling laws for neural language models,” *arXiv preprint arXiv:2001.08361*, 2020. [Online]. Available: <https://arxiv.org/abs/2001.08361>
- [5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, 2020. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [6] G. Izacard and E. Grave, “Leveraging passage retrieval with generative models for open domain question answering,” in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, 2021, pp. 874–880. [Online]. Available: <https://arxiv.org/abs/2007.01282>
- [7] S. Borgeaud, A. Mensch, J. Hoffmann *et al.*, “Improving language models by retrieving from trillions of tokens,” in *International Conference on Machine Learning*, 2022. [Online]. Available: <https://arxiv.org/abs/2112.04426>
- [8] J. Dongarra, P. Beckman, T. Moore, P. Aerts *et al.*, “High-performance computing: Challenges and opportunities,” *International Journal of High Performance Computing Applications*, 2020. [Online]. Available: <https://doi.org/10.1177/1094342020957393>

- [9] G. M. Kurtzer, V. Sochat, and M. W. Bauer, “Singularity: Scientific containers for mobility of compute,” *PLOS ONE*, vol. 12, no. 5, 2017. [Online]. Available: <https://doi.org/10.1371/journal.pone.0177459>
- [10] A. B. Yoo, M. A. Jette, and M. Grondona, “Slurm: Simple linux utility for resource management,” in *Job Scheduling Strategies for Parallel Processing*. Springer, 2003. [Online]. Available: https://doi.org/10.1007/10968987_3
- [11] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, “Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing,” *arXiv preprint arXiv:2107.13586*, 2023. [Online]. Available: <https://arxiv.org/abs/2107.13586>
- [12] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” *arXiv preprint arXiv:2106.09685*, 2022. [Online]. Available: <https://arxiv.org/abs/2106.09685>
- [13] J. Pfeiffer, A. Kamath, A. Ruckle, K. Cho, and I. Gurevych, “Adapterfusion: Non-destructive task composition for transfer learning,” in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, 2021, pp. 487–503. [Online]. Available: <https://aclanthology.org/2021.eacl-main.39/>
- [14] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: Bm25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009. [Online]. Available: <https://doi.org/10.1561/15000000019>
- [15] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020, pp. 6769–6781. [Online]. Available: <https://aclanthology.org/2020.emnlp-main.550/>
- [16] J. Johnson, M. Douze, and H. Jegou, “Billion-scale similarity search with gpus,” *IEEE Transactions on Big Data*, 2017. [Online]. Available: <https://ieeexplore.ieee.org/document/8733051>
- [17] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world

- graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020. [Online]. Available: <https://ieeexplore.ieee.org/document/8626655>
- [18] R. Nogueira and K. Cho, “Passage re-ranking with bert,” in *arXiv preprint arXiv:1901.04085*, 2019. [Online]. Available: <https://arxiv.org/abs/1901.04085>
- [19] O. Khattab and M. Zaharia, “Colbert: Efficient and effective passage search via contextualized late interaction over bert,” in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020, pp. 39–48. [Online]. Available: <https://dl.acm.org/doi/10.1145/3397271.3401075>
- [20] C.-Y. Lin, “ROUGE: A package for automatic evaluation of summaries,” in *Text Summarization Branches Out*. Association for Computational Linguistics, 2004, pp. 74–81. [Online]. Available: <https://aclanthology.org/W04-1013/>
- [21] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating text generation with BERT,” in *International Conference on Learning Representations*, 2020. [Online]. Available: <https://openreview.net/forum?id=SkeHuCVFDr>
- [22] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019, pp. 3982–3992. [Online]. Available: <https://aclanthology.org/D19-1410/>
- [23] W. Zhang, C. Li, and A. Kumar, “A survey on machine learning system design: Performance, scalability, and efficiency,” *ACM Computing Surveys*, 2021. [Online]. Available: <https://doi.org/10.1145/3439723>
- [24] P. Liang, R. Bommasani *et al.*, “Holistic evaluation of language models,” *arXiv preprint arXiv:2211.09110*, 2022. [Online]. Available: <https://arxiv.org/abs/2211.09110>
- [25] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “Ragas: Automated evaluation of retrieval augmented generation,” *EACL 2024 - 18th Conference of the European Chapter of the Association for Computational Linguistics, Proceedings of*

- System Demonstrations*, pp. 150–158, 2024. [Online]. Available: <https://aclanthology.org/2024.eacl-demo.16/>
- [26] D. Narayanan, K. Santhanam, A. Phanishayee, and M. Zaharia, “Efficient large-scale language model serving on cloud platforms,” *arXiv preprint arXiv:2109.00053*, 2021. [Online]. Available: <https://arxiv.org/abs/2109.00053>
- [27] M. Shoeybi, M. Patwary, R. Puri, P. Le *et al.*, “Megatron-lm: Training multi-billion parameter language models using model parallelism,” *arXiv preprint arXiv:1909.08053*, 2019. [Online]. Available: <https://arxiv.org/abs/1909.08053>
- [28] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, “Zero: Memory optimization towards training trillion parameter models,” *arXiv preprint arXiv:1910.02054*, 2020. [Online]. Available: <https://arxiv.org/abs/1910.02054>
- [29] W. Kwon, B. Y. Li, Y. J. Lee, D. Patterson, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” *arXiv preprint arXiv:2309.06180*, 2023. [Online]. Available: <https://arxiv.org/abs/2309.06180>
- [30] T. Dettmers, M. Lewis, S. Shleifer, and L. Zettlemoyer, “Llm.int8(): 8-bit matrix multiplication for transformers at scale,” *arXiv preprint arXiv:2208.07339*, 2022. [Online]. Available: <https://arxiv.org/abs/2208.07339>
- [31] J. Frankle and M. Carbin, “The lottery ticket hypothesis: Finding sparse, trainable neural networks,” in *Proceedings of the International Conference on Learning Representations*, 2019. [Online]. Available: <https://arxiv.org/abs/1803.03635>
- [32] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter,” in *Proceedings of the 5th Workshop on Energy Efficient Machine Learning and Cognitive Computing*, 2019. [Online]. Available: <https://arxiv.org/abs/1910.01108>
- [33] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” in *Advances in Neural Information Processing Systems*, 2022. [Online]. Available: <https://arxiv.org/abs/2205.14135>

- [34] H. Chase, “Langchain: Building applications with large language models,” *Technical Report*, 2023. [Online]. Available: <https://arxiv.org/abs/2307.11536>
- [35] J. Liu, “Llamaindex: A data framework for llm applications,” *Technical Report*, 2023. [Online]. Available: <https://arxiv.org/abs/2308.07862>
- [36] A. Abdolrashidi, P. Mattson, C.-J. Wu, Y. Cheng *et al.*, “Large-scale evaluation of deep learning workloads on hpc platforms,” in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 883–895.
- [37] V. J. Reddi, C. Cheng *et al.*, “Mlperf inference benchmark,” in *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture*, 2020. [Online]. Available: <https://arxiv.org/abs/1911.02549>
- [38] High Performance Computing Facility, “Cyclone supercomputer infrastructure,” <https://hpcf.cyi.ac.cy/>, The Cyprus Institute.

Appendices

Appendix A: Experimental Artifacts and Reproducibility Checklist

This appendix summarises the main artifacts used to reproduce the experiments described in chapters 4–6. The purpose is not to repeat the methodology chapter, but to provide a compact reference for the files, notebooks, result logs, and validation steps that were used during the local and HPC stages of the work.

A.1 Project Artifact Map

Artifact	Role in the thesis workflow
data/dummy_data/	Synthetic OrchestrAI enterprise corpus used by all experiments. The corpus contains 54 DOCX files grouped by department.
notebooks/	Local experiment notebooks using Ollama for generation and embeddings, and Milvus where vector-database retrieval is required.
notebooks-hpc/	HPC experiment notebooks with the same six pipeline structures adapted for vLLM-based services.
results_logger.py	Shared CSV logger that records the implementation name, scenario identifier, question, generated answer, retrieved sources, retrieved contexts, top-k values, and timing fields.
ground_truth_builder.py	Helper script for creating a manual ground-truth worksheet from generated results.
deterministic_eval.py	Evaluation script that merges results with manual ground truth and computes answer-quality and retrieval-quality metrics.
rag_ground_truth.csv	Manual evaluation file containing reference answers, supporting documents, manual scores, labels, and notes.
hpc_results.csv	HPC timing output used for the local-versus-HPC comparison.
images/	Figures and plots inserted into the thesis, including quality charts, timing charts, and speed-up visualisations.

Table 7.2: Main artifacts used to reproduce and document the thesis experiments.

A.2 Pipeline Summary

Notebook	Purpose
<code>rag_basic.ipynb</code>	In-memory dense retrieval baseline with no Milvus dependency.
<code>rag_vectordb.ipynb</code>	Dense retrieval using Milvus as a persistent vector database.
<code>rag_vectordb_tuned.ipynb</code>	Milvus-backed dense retrieval with larger chunks and larger overlap.
<code>rag_reranker.ipynb</code>	Milvus dense retrieval followed by cross-encoder reranking.
<code>rag_hybrid.ipynb</code>	Milvus hybrid retrieval combining dense semantic search and sparse lexical matching.
<code>rag_hybrid_&_rerank.ipynb</code>	Hybrid retrieval followed by reranking before answer generation.

Table 7.3: Implemented pipeline variants and their corresponding notebooks.

A.3 Scenario Set

Scenario	Question	Primary retrieval behaviour tested
1	What is the first-response SLA for a Sev-1 support incident?	Single-document factual lookup.
2	What steps must be completed before a new employee is considered ready for day one at OrchestrAI?	Checklist and procedure retrieval.
3	A new engineer starts on Monday, but their laptop has not arrived and their required access is still pending. Which teams are involved and what should happen next?	Cross-team HR, IT, and access reasoning.
4	Can OrchestrAI make an urgent hardware purchase without following the normal PO workflow if a warehouse outage is at risk?	Policy exception reasoning.
5	A customer renewal is at risk because support cases are recurring, a requested feature is still unavailable, and replacement hardware is delayed. What should OrchestrAI do next, and which teams must be involved?	Multi-hop customer-risk analysis.

Table 7.4: Scenario questions used across all six pipeline implementations.

A.4 Logged Fields

Each notebook writes rows using the shared logging schema below. This keeps local and HPC outputs comparable at the workflow level.

- **implementation:** pipeline identifier, such as `basic`, `vectordb`, or `hybrid_rerank`.
- **scenario_id:** scenario identifier used to distinguish the five questions per implementation.
- **question:** input question sent to the RAG pipeline.
- **generated_answer:** answer produced by the language model.
- **retrieved_sources:** filenames of retrieved source documents.

- `retrieved_departments`: department metadata associated with the retrieved documents.
- `retrieved_contexts`: retrieved text fragments passed into the prompt.
- `num_retrieved_docs`: number of retrieved documents or chunks.
- `retriever_top_k` and `reranker_top_k`: retrieval and reranking limits used by the implementation.
- `indexing_time_s`, `retrieval_time_s`, and `generation_time_s`: application-level timing measurements in seconds.

A.5 Reproducibility Checklist

Before using a new run for analysis, the following checks should be completed:

1. Confirm that the same 54 DOCX files are available under `data/dummy_data/`.
2. Confirm that Ollama, Milvus, or the HPC vLLM services are running before executing notebooks that depend on them.
3. Run the six notebooks in the documented order: basic, vector database, tuned vector database, reranker, hybrid, and hybrid with reranking.
4. Verify that each implementation produces five result rows, one for each scenario.
5. Verify that no result row has an empty generated answer, empty retrieved source list, or missing timing field.
6. Build or update the manual ground-truth worksheet using evidence from the DOCX corpus only.
7. Run the deterministic evaluation script after the manual scores, supporting documents, and reference answers have been completed.
8. Copy only curated final results and figures into the final result directories used by the thesis.

Appendix B: Evaluation Rubric and Supporting Evidence Guide

This appendix records the manual scoring logic used alongside the deterministic evaluation script. The manual assessment was necessary because automatic similarity metrics such as ROUGE-L, BERTScore, and sentence-embedding similarity can identify overlap with a reference answer, but they do not always determine whether an answer contains the operational detail required by the scenario.

B.1 Manual Answer Score

Score	Label	Interpretation
1.0	correct	The answer is supported by the corpus and includes the required operational facts or steps for the scenario. Minor wording differences are acceptable.
0.5	partially_correct	The answer contains some relevant and supported information, but omits required details, mixes responsibilities, or does not fully answer the question.
0.0	incorrect	The answer is unsupported, contradicts the corpus, retrieves the wrong operational process, or fails to answer the scenario.

Table 7.5: Manual answer scoring rubric used in the thesis evaluation.

B.2 Supporting Document Rules

Supporting documents are recorded in `data/ground_truth/rag_ground_truth.csv`. The following rules were used when filling the manual evidence fields:

- Supporting documents must be selected from the DOCX corpus only.
- The `supporting_documents` field should contain filenames that directly justify the reference answer.

- If a scenario requires more than one department, all essential supporting files should be recorded.
- Manual notes should explain omissions, unsupported statements, or incorrect operational reasoning.
- The score should be based on correctness against the corpus, not on whether the generated answer sounds fluent.

B.3 Deterministic Evaluation Outputs

After the manual worksheet is completed, the deterministic evaluation script produces per-row and per-implementation outputs. The key metrics are:

- **Answer quality:** manual score, ROUGE-L F1, BERTScore F1, sentence-embedding semantic similarity, and answer length.
- **Retrieval quality:** retrieval hit, retrieval precision, retrieval recall, and retrieval F1.
- **Timing:** indexing time, retrieval time, and generation time averaged by implementation.

The local final outputs used in the thesis are stored in `data/results/final/`. The HPC comparison uses timing rows generated by the `notebooks-hpc/` workflow. Since the HPC phase was used for timing comparison only, the main answer-quality and retrieval-quality interpretation is based on the local evaluation workflow.

Appendix C: Supplementary Figures

This appendix includes supplementary figures generated during the project. They support the implementation and timing discussion but are kept outside the main chapters so that the core results chapter remains focused on the most important comparisons.

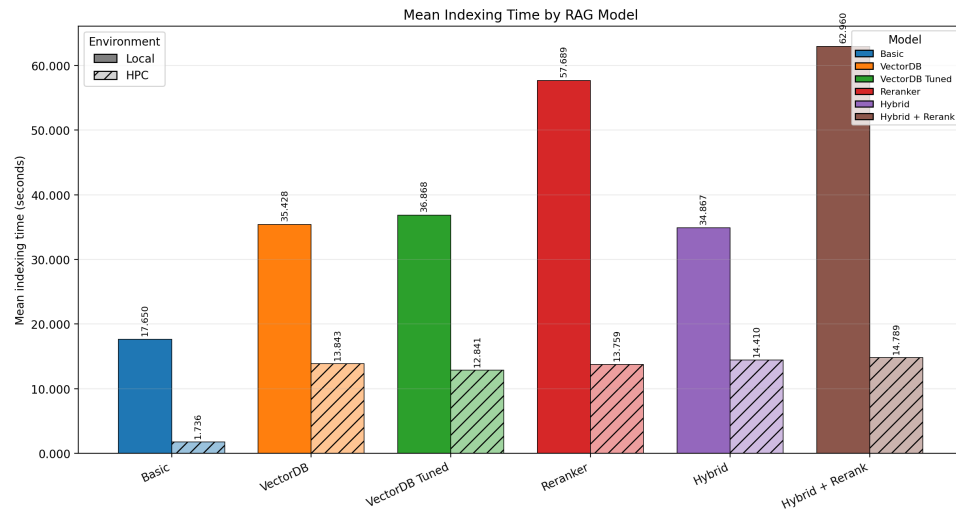


Figure 7.1: Mean indexing time by pipeline and execution environment.

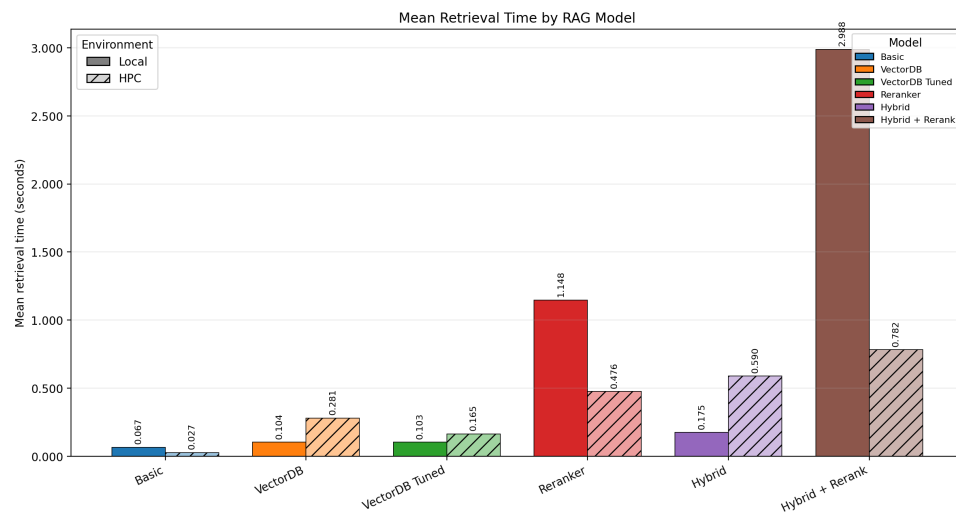


Figure 7.2: Mean retrieval time by pipeline and execution environment.

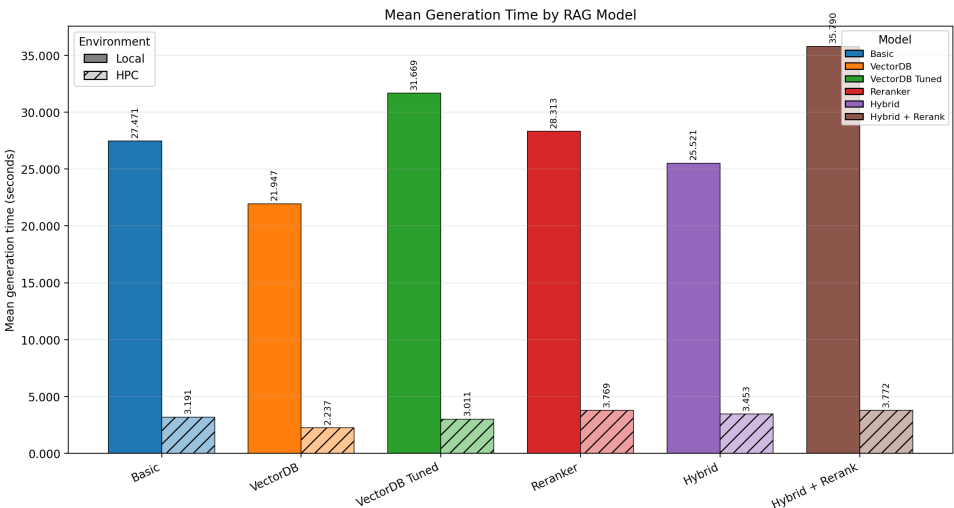


Figure 7.3: Mean generation time by pipeline and execution environment.

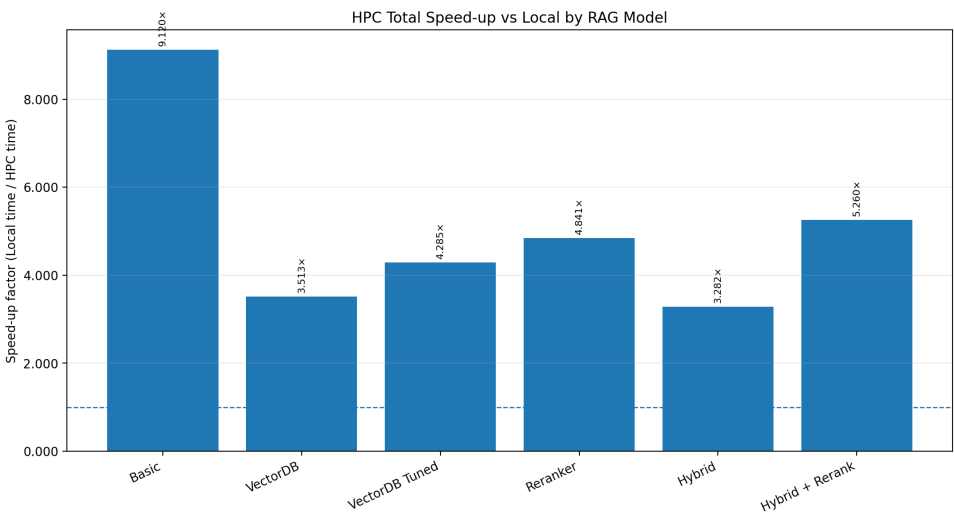


Figure 7.4: Total speed-up factor for each RAG pipeline when comparing local and HPC execution.

Manual

Installation Manual

Purpose and Scope

This manual explains how to prepare the software environment used by the thesis experiments. The project contains two execution paths: a local workflow using Jupyter, Ollama, Docker, and Milvus; and an HPC workflow using Slurm-managed vLLM services. The local workflow is used for the main retrieval-quality and answer-quality evaluation. The HPC workflow is used for the timing comparison described in Chapters 5 and 6.

Required Software for Local Execution

The local workflow requires the following software:

- Python 3.12 or a compatible Python 3 version supported by the project dependencies;
- Jupyter Notebook or JupyterLab;
- Docker, used to run Milvus in standalone mode;
- Ollama, used to serve the local generation and embedding models;
- the Python packages listed in `requirements.txt`.

The local models required through Ollama are:

- `llama3.1` or `llama3.1:8b` for answer generation;
- `mxbai-embed-large` for embeddings.

Local Installation Steps

1. Open a terminal in the project root directory.

2. Create and activate a virtual environment:

```
python3.12 -m venv .venv
source .venv/bin/activate
```

3. Install the Python dependencies:

```
python -m pip install -upgrade pip
python -m pip install -r requirements.txt
```

4. Start the Ollama service in a separate terminal:

```
ollama serve
```

5. Download the required local models:

```
ollama pull llama3.1
ollama pull mxbai-embed-large
```

6. Start Milvus before running any vector-database, reranking, hybrid, tuned-vector, or hybrid-reranking notebook:

```
bash scripts/standalone_embed.sh start
```

7. Start Jupyter:

```
jupyter notebook
```

The in-memory baseline notebook does not require Milvus. All other local pipeline notebooks require Milvus to be running.

Milvus Management

The project includes `scripts/standalone_embed.sh` for managing the local Milvus container. The main commands are:

- Start Milvus: `bash scripts/standalone_embed.sh start`
- Stop Milvus: `bash scripts/standalone_embed.sh stop`
- Restart Milvus: `bash scripts/standalone_embed.sh restart`
- Delete the Milvus container and local Milvus data: `bash scripts/standalone_embed.sh delete`

The delete command removes local Milvus runtime data and should only be used when a clean vector-database state is required.

HPC Installation and Service Preparation

The HPC workflow is designed for the Cyclone cluster environment. It uses Slurm to launch model-serving jobs and Jupyter. The user should first log in to the cluster:

```
ssh <username>@front01.hpcf.cyi.ac.cy
```

If the generation model requires Hugging Face access, create a local environment file on the cluster and store the token there. The token must not be committed to version control:

```
cd ~/Thesis
cp .env.example .env
chmod 600 .env
```

Then edit `.env` and set `HF_TOKEN`. The HPC service launch scripts read this file when needed.

The vLLM services used by the HPC workflow are:

- generation service: `meta-llama/Llama-3.1-8B-Instruct`;
- embedding service: `BAAI/bge-m3`;
- reranking service: `BAAI/bge-reranker-base`.

To start the services, run the service launcher from the vLLM launcher directory:

```
cd ~/Thesis/rag-training2025-main/vllm
bash serve_models.sh
```

The launcher submits separate Slurm jobs for the language model, embedding model, and reranking model. Each service writes connection information to environment files under `/nvme/scratch/${USER}` so that the HPC notebooks can connect to the correct endpoints.

To start Jupyter on the cluster:

```
cd ~/Thesis/scripts
sbatch launch_jupyter.sh
```

After the job starts, use the generated connection information to create the SSH tunnel and open the Jupyter URL from the local browser.

User Manual

Project Workflow

The project is organised around a fixed experiment sequence. Each pipeline uses the same OrchestrAI document corpus and the same five scenario questions. The main difference between notebooks is the retrieval strategy.

Order	Notebook	Use
1	<code>rag_basic.ipynb</code>	Run the in-memory dense retrieval baseline.
2	<code>rag_vectordb.ipynb</code>	Run dense retrieval using Milvus.
3	<code>rag_vectordb_tuned.ipynb</code>	Run Milvus dense retrieval with larger chunks and overlap.
4	<code>rag_reranker.ipynb</code>	Run dense retrieval followed by reranking.
5	<code>rag_hybrid.ipynb</code>	Run hybrid dense and sparse retrieval.
6	<code>rag_hybrid_&_rerank.ipynb</code>	Run hybrid retrieval followed by reranking.

Table 7.6: Recommended notebook execution order.

Running the Local Experiments

1. Start Ollama and confirm the required models are available.
2. Start Milvus if the notebook is not the in-memory baseline.
3. Open Jupyter from the project root.
4. Open the required notebook from `notebooks/`.
5. Run the cells from top to bottom without changing the scenario questions unless a new experiment is being designed.
6. Confirm that the notebook writes rows to `data/results/experimental/rag_results`.
7. Repeat the process for all six notebooks in the order shown in Table 7.6.

Each notebook clears or replaces previous rows for its own implementation before writing new rows. This prevents duplicate rows for the same implementation when a notebook is rerun.

Running the HPC Experiments

1. Log in to the HPC system.
2. Start the vLLM generation, embedding, and reranking services.
3. Submit the Jupyter Slurm job and connect through the SSH tunnel.
4. Open the notebooks from `notebooks-hpc/`.
5. Run the same six notebook variants in the same order used locally.
6. Confirm that timing rows are written to the HPC results file.

The HPC phase is used for workflow-level timing comparison. It should not be interpreted as a pure hardware-only benchmark because the local and HPC workflows use different serving stacks and some different backend models.

Manual Ground-Truth Evaluation

After local result rows are generated, the manual evaluation file should be completed in `data/ground_truth/rag_ground_truth.csv`. The evaluator should inspect the DOCX corpus and fill:

- `ground_truth_answer`: reference answer supported by the corpus;
- `supporting_documents`: filenames used as evidence;
- `manual_score`: 1.0, 0.5, or 0.0;
- `manual_label`: correct, partially_correct, or incorrect;
- `manual_notes`: short explanation of the score.

The scoring should be strict. A fluent answer should not receive a high score unless it is supported by the corpus and answers the scenario completely.

Running Deterministic Evaluation

After the manual ground-truth file is complete, run:

```
python3 src/deterministic_eval.py
```

This produces:

- `data/results/experimental/deterministic_eval_results.csv`
- `data/results/experimental/deterministic_eval_summary.csv`

The summary file averages answer-quality metrics, retrieval-quality metrics, and timing fields by implementation. These outputs are used to create the tables and figures in the results chapter.

Generating Figures

The local visualisation notebook is `notebooks/rag_visualizations.ipynb`. It reads the evaluated outputs and generates charts for answer quality, retrieval quality, timing, scenario-level performance, and multi-metric comparison.

The HPC visualisation workflow reads local and HPC timing outputs and produces figures for mean timing, speed-up factors, heatmaps, and stacked runtime components. Final thesis figures are placed in `Documentation/images/` before compilation.

Result Validation Checklist

Before using a run in the thesis, verify that:

- each of the six implementations has five scenario rows;
- every row has a generated answer;
- every row has retrieved source information;
- `indexing_time_s`, `retrieval_time_s`, and `generation_time_s` are present;

- the manual ground-truth file contains supporting documents and notes for every evaluated row;
- deterministic evaluation results were regenerated after any manual scoring changes;
- only curated final outputs are copied into `data/results/final/` and `Documentation/images/`.

Common Issues

- If Milvus notebooks fail to connect, confirm Docker is running and restart Milvus with `bash scripts/standalone_embed.sh restart`.
- If Ollama generation or embedding calls fail, confirm `ollama serve` is running and that the required models have been pulled.
- If deterministic evaluation fails, confirm that both the results CSV and ground-truth CSV contain matching `implementation` and `scenario_id` values.
- If HPC notebooks cannot reach a model service, confirm that the Slurm service jobs are running and that the environment files under `/nvme/scratch/${USER}` exist.
- If repeated notebook runs create unexpected results, clear or inspect the implementation rows in the relevant results CSV before rerunning the notebook.